```
#ML LABORATORY
# WEEK 1: Implement Naïve Bayes classifier and k-Nearest Neighbor (k-
NN)

# WEEK 2: Implement Decision trees – ID3 & CART

# WEEK 3: Implement Linear Regression learning algorithm and logistic
Regression learning algorithm.

# WEEK 4: Implement Support vector machines learning algorithm- Linear
and Non -Linear

# WEEK 5: Implement unsupervised k-means algorithm

# WEEK 6: Apply EM algorithm to cluster a set of data stored in a .CSV
file. Use the same dataset for clustering using k-Means algorithm.
Compare the results of these two algorithms and comment on the quality
of clustering.

# WEEK 7: Model Training: Holdout, K-Fold cross validation and
Bootstrap Sampling

# WEEK 8: Evaluating Model Performance:
# Supervised Learning- Classification
# Supervised Learning- Regression
# Unsupervised Learning-Clustering
# (Use Lower Back Pain Symptoms dataset available at Kaggle
# (https://www.kaggle.com/sammy123/lower-back-pain- symptoms-dataset).
The dataset spint.csv consists of 310 observations and 13 attributes).
Also identify the cluster quality with purity and silhouette width.

# Week 9: Implement Random Forest model learning algorithm

# WEEK 10: Implement Adaboost, XGboost algorithms.

# WEEK 11: Implement single layer perceptron algorithm and multilayer
perceptron algorithm

#KNN as classifier  week 1
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import matplotlib.pyplot as plt

data = load_breast_cancer()
X = data.data
```

```python
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

k = 5
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print(f"Accuracy:{accuracy_score(y_test, y_pred)* 100:.2f}%")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

# Print predictions
print("Predicted Labels:", y_pred[:10])  # First 10 predictions
print("Actual Labels:", y_test[:10])

k_values = range(1, 21)
accuracies = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred_k = knn.predict(X_test)
    accuracies.append(accuracy_score(y_test, y_pred_k))

plt.plot(k_values, accuracies, marker='o')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Accuracy')
plt.title('k-NN Varying k')
plt.grid(True)
plt.show()
```

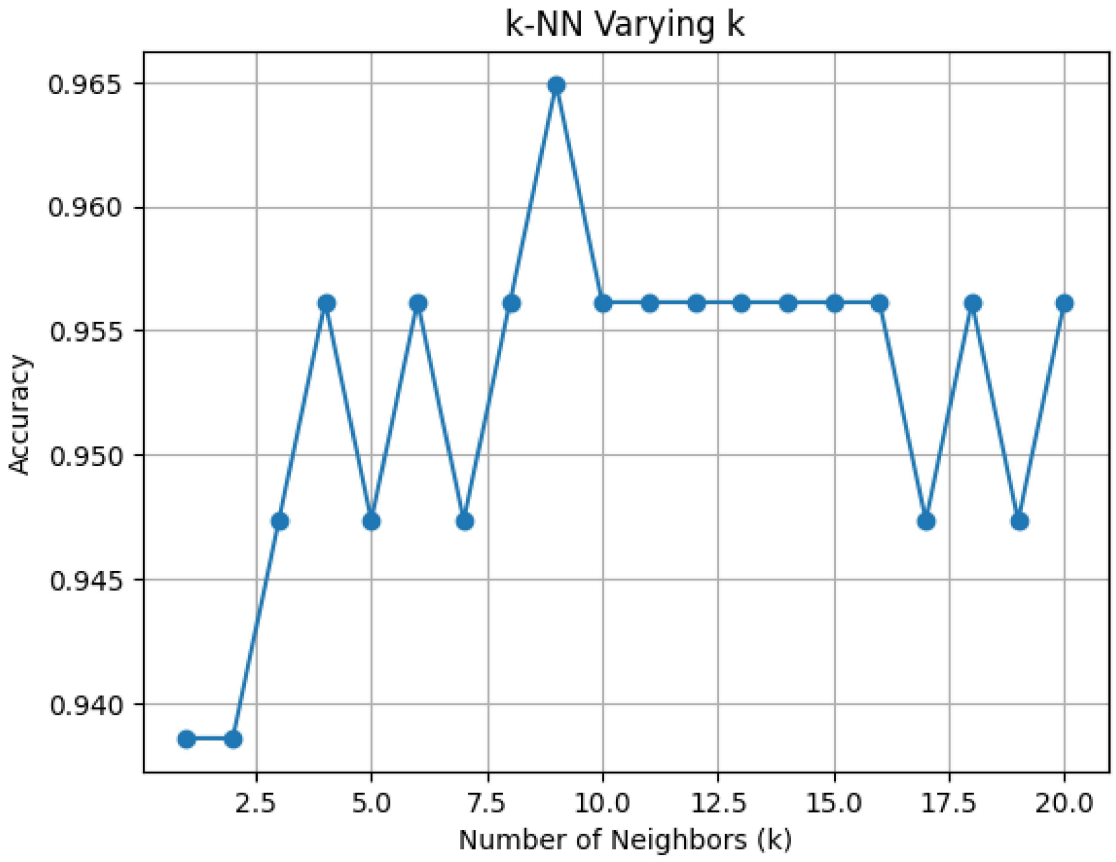
```
Accuracy:94.74%
              precision    recall  f1-score   support

           0       0.93      0.93      0.93        43
           1       0.96      0.96      0.96        71

    accuracy                           0.95       114
   macro avg       0.94      0.94      0.94       114
weighted avg       0.95      0.95      0.95       114

[[40  3]
 [ 3 68]]
```

```
Predicted Labels: [1 0 0 1 1 0 0 0 0 1]
Actual Labels: [1 0 0 1 1 0 0 0 1 1]
```




```
#knn as regressor week 1
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

url =
"https://raw.githubusercontent.com/selva86/datasets/master/BostonHousi
ng.csv"
data = pd.read_csv(url)

X = data.drop('medv', axis=1).values  # Features
y = data['medv'].values  # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y,
```

```
test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

k = 5
knn_regressor = KNeighborsRegressor(n_neighbors=k)
knn_regressor.fit(X_train, y_train)

y_pred = knn_regressor.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse:.2f}")
print(f"R² Score: {r2:.2f}")

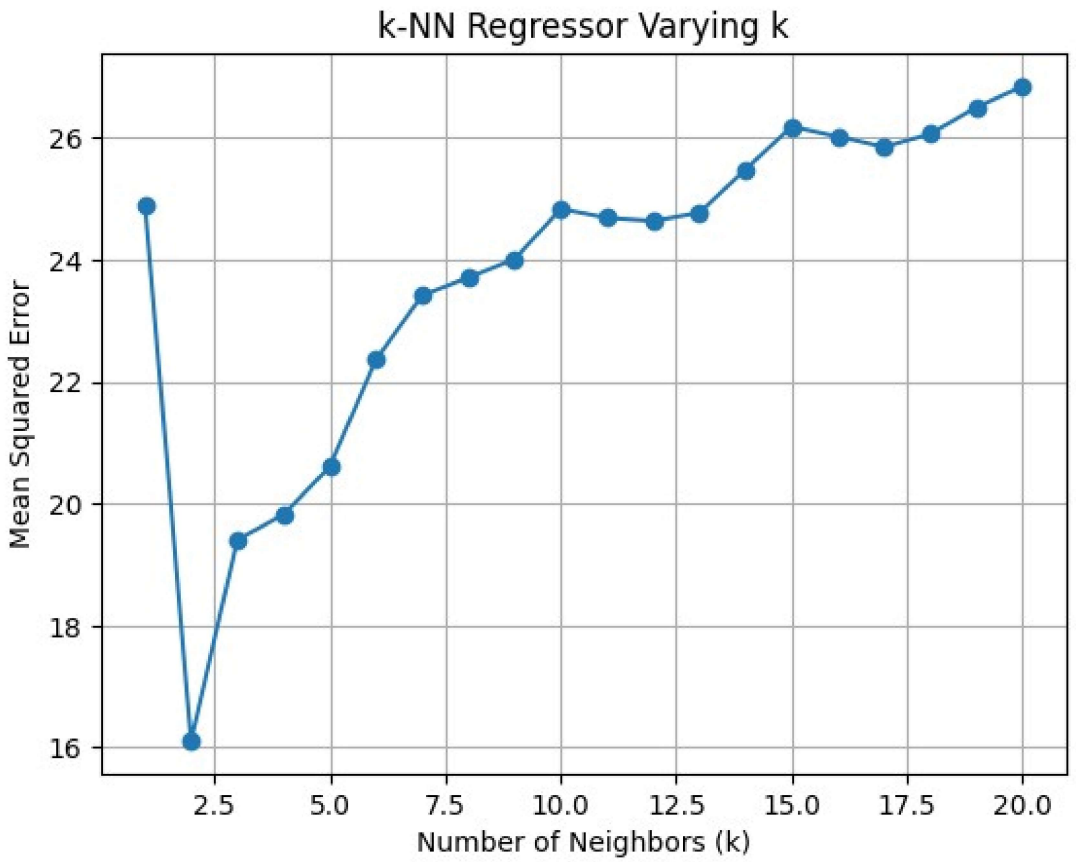
print("Predicted Values:", y_pred[:10])
print("Actual Values:", y_test[:10])

k_values = range(1, 21)
mse_values = []

for k in k_values:
    knn_regressor = KNeighborsRegressor(n_neighbors=k)
    knn_regressor.fit(X_train, y_train)
    y_pred_k = knn_regressor.predict(X_test)
    mse_values.append(mean_squared_error(y_test, y_pred_k))

plt.plot(k_values, mse_values, marker='o')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Mean Squared Error')
plt.title('k-NN Regressor Varying k')
plt.grid(True)
plt.show()
```

```
Mean Squared Error: 20.61
R² Score: 0.72
Predicted Values: [25.84 33.64 17.34 24.08 17.44 22.   19.64 16.18
19.32 19.58]
Actual Values: [23.6 32.4 13.6 22.8 16.1 20.  17.8 14.  19.6 16.8]
```




```
#multinomial Navie Bayes week 1
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

df = pdf = pd.read_csv("C:/Users/shanm/Downloads/archive (1)/breast-
cancer.csv")
x = df.drop(['diagnosis'], axis=1)
y = df['diagnosis']

x_minmaxscalered = MinMaxScaler().fit_transform(x)
x_train, x_test, y_train, y_test = train_test_split(x_minmaxscalered,
y, test_size=0.33, random_state=125)

model = MultinomialNB()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print(f"Accuracy:{accuracy_score(y_test, y_pred)* 100:.2f}%")
```

```
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))


Accuracy:82.45%
              precision    recall  f1-score   support

           B       0.79      0.99      0.88       118
           M       0.97      0.54      0.70        70

    accuracy                           0.82       188
   macro avg       0.88      0.77      0.79       188
weighted avg       0.86      0.82      0.81       188

[[117   1]
 [ 32  38]]

#Gaussian Navie Bayes   week 1
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.model selection import train test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
data = load_iris()
x = data.data
y = data.target

x_scaled = StandardScaler().fit_transform(x)

x_train, x_test, y_train, y_test = train_test_split(x_scaled, y,
test_size=0.33, random_state=125)

model = GaussianNB()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy * 100:.2f}%")
print("\nConfusion Matrix:")
print(conf_matrix)
print("\nClassification Report:")
print(class_report)

Accuracy: 98.00%
```

```
Confusion Matrix:
[[18  0  0]
 [ 0 14  1]
 [ 0  0 17]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        18
           1       1.00      0.93      0.97        15
           2       0.94      1.00      0.97        17

    accuracy                           0.98        50
   macro avg       0.98      0.98      0.98        50
weighted avg       0.98      0.98      0.98        50
```

```python
#Bernoulli Navie Bayes week 1
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import Binarizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

data = load_breast_cancer()
X = data.data
y = data.target

# Binarize features based on the median value for each feature
X_binarized = (X > np.median(X, axis=0)).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X_binarized, y,
test_size=0.33, random_state=125)

model = BernoulliNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f'Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%')
print('Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
print('Classification Report:')
print(classification_report(y_test, y_pred))
```

```
Accuracy: 91.49%
Confusion Matrix:
[[ 64   6]
 [ 10 108]]
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.91      0.89        70
           1       0.95      0.92      0.93       118

    accuracy                           0.91       188
   macro avg       0.91      0.91      0.91       188
weighted avg       0.92      0.91      0.92       188
```

```
#multinomial Navie Bayes   week1
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

data = load_breast_cancer()
X = data.data
y = data.target

# Scale the features to make them suitable for MultinomialNB
scaler = MinMaxScaler()  # Min-Max scaling to ensure non-negative
values
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

mnb = MultinomialNB()
mnb.fit(X_train, y_train)
y_pred = mnb.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 83.33%
              precision    recall  f1-score   support

           0       1.00      0.56      0.72        43
           1       0.79      1.00      0.88        71

    accuracy                           0.83       114
   macro avg       0.89      0.78      0.80       114
weighted avg       0.87      0.83      0.82       114

[[24 19]
 [ 0 71]]
```

```python
#WEEK 2: Implement Decision trees – ID3 & CART
#ID3
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_wine
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

# Load the Wine dataset
wine = load_wine()
df = pd.DataFrame(data=wine.data, columns=wine.feature_names)
df['target'] = wine.target

# Print the first few rows of the dataset
print(df.head())

# Split the data into features and target
X = df.drop(columns=['target'])
y = df['target']

# Initialize the DecisionTreeClassifier with the ID3 algorithm (using
'entropy' criterion)
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X, y)

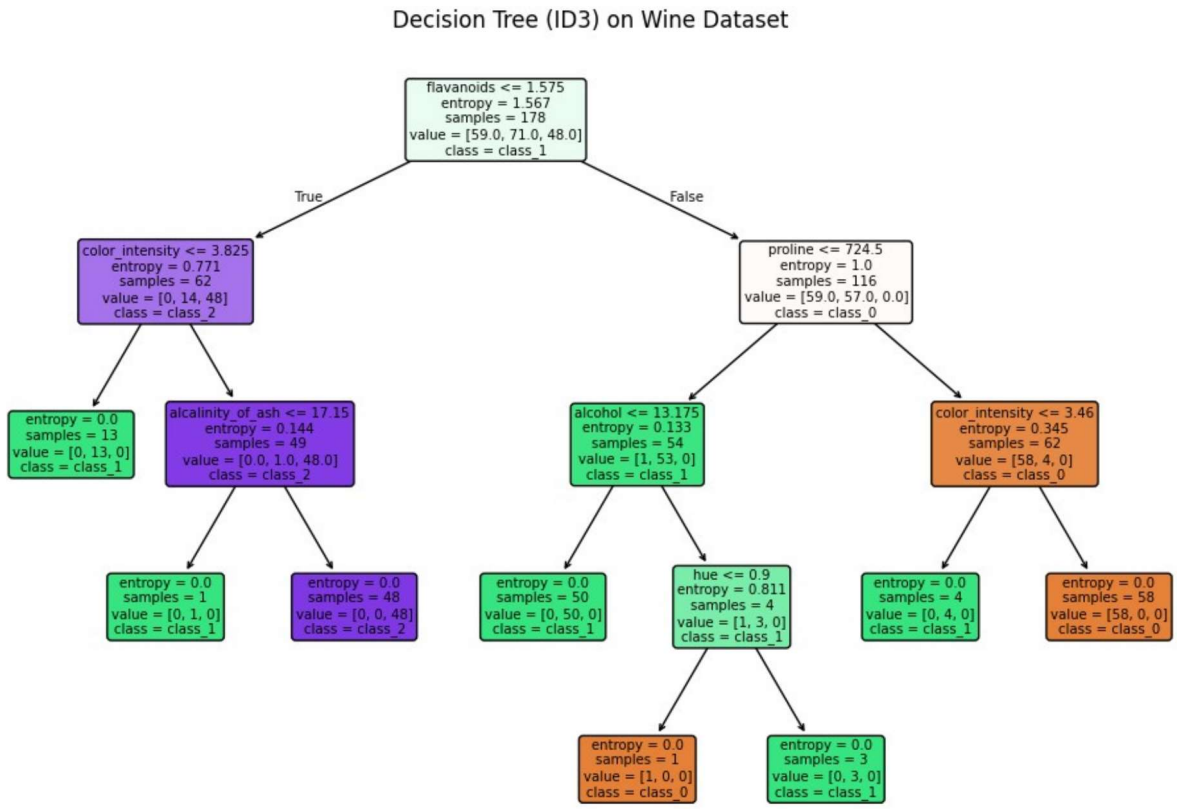
# Plot the decision tree
plt.figure(figsize=(12, 8))
plot tree(model, feature names=wine.feature names,
class_names=wine.target_names, filled=True, rounded=True)
plt.title("Decision Tree (ID3) on Wine Dataset")
plt.show()
```

```
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium
total_phenols  \
0    14.23        1.71  2.43               15.6      127.0
2.80
1    13.20        1.78  2.14               11.2      100.0
2.65
2    13.16        2.36  2.67               18.6      101.0
2.80
3    14.37        1.95  2.50               16.8      113.0
3.85
4    13.24        2.59  2.87               21.0      118.0
2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity
hue  \
0        3.06                  0.28             2.29             5.64
```

```
1.04
1        2.76                    0.26                1.28                4.38
1.05
2        3.24                    0.30                2.81                5.68
1.03
3        3.49                    0.24                2.18                7.80
0.86
4        2.69                    0.39                1.82                4.32
1.04
```

	od280/od315 of diluted wines	proline	target
0	3.92	1065.0	0
1	3.40	1050.0	0
2	3.17	1185.0	0
3	3.45	1480.0	0
4	2.93	735.0	0




```python
#cart classifier week 2
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_wine
import matplotlib.pyplot as plt
```

```
# Load the Wine dataset
wine = load_wine()
df = pd.DataFrame(data=wine.data, columns=wine.feature_names)
df['target'] = wine.target

# Print the first few rows of the dataset
print(df.head())

# Split the data into features and target
X = df.drop(columns=['target'])
y = df['target']

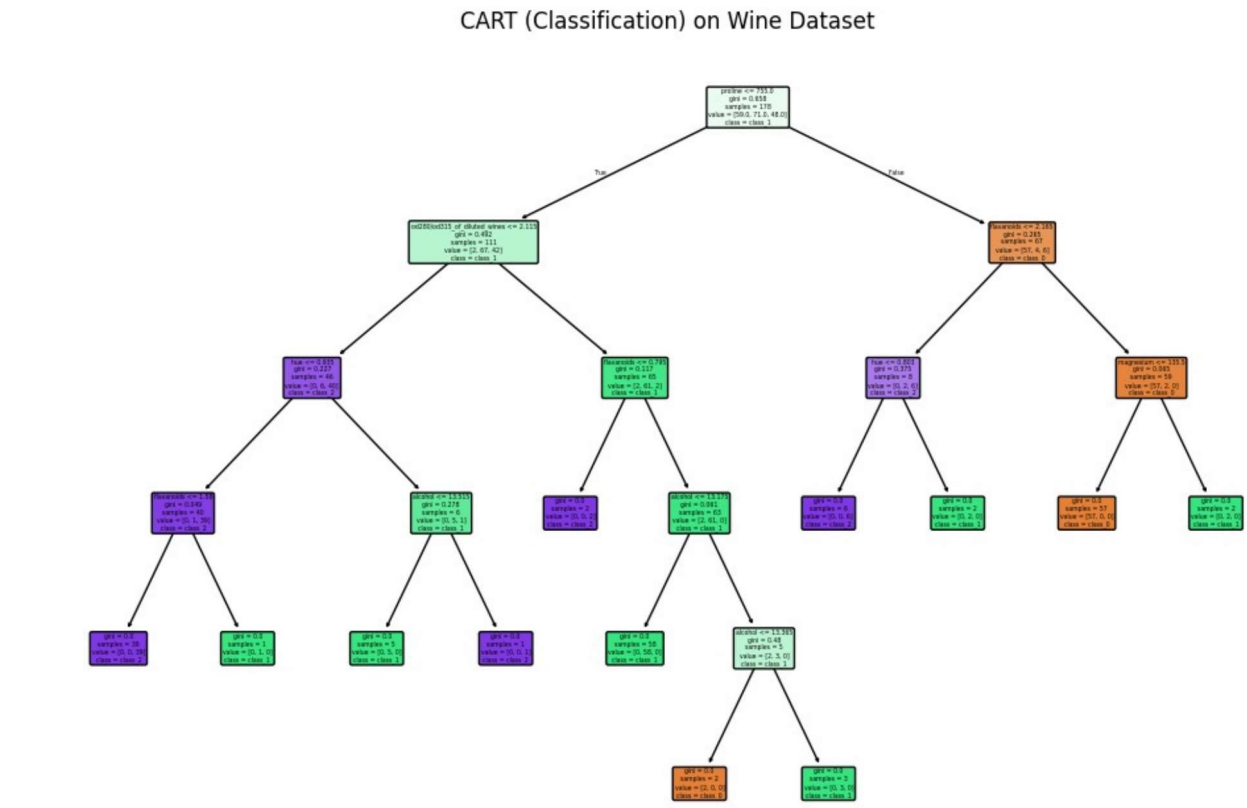
# Initialize the DecisionTreeClassifier with the CART algorithm (using
'gini' criterion)
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X, y)

# Plot the decision tree
plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=wine.feature_names,
class_names=wine.target_names, filled=True, rounded=True)
plt.title("CART (Classification) on Wine Dataset")
plt.show()
```

```
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium
total_phenols  \
0    14.23        1.71  2.43               15.6      127.0
2.80
1    13.20        1.78  2.14               11.2      100.0
2.65
2    13.16        2.36  2.67               18.6      101.0
2.80
3    14.37        1.95  2.50               16.8      113.0
3.85
4    13.24        2.59  2.87               21.0      118.0
2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity
hue  \
0        3.06                  0.28             2.29             5.64
1.04
1        2.76                  0.26             1.28             4.38
1.05
2        3.24                  0.30             2.81             5.68
1.03
3        3.49                  0.24             2.18             7.80
0.86
4        2.69                  0.39             1.82             4.32
1.04
```

```
   od280/od315_of_diluted_wines  proline  target
0                          3.92   1065.0       0
1                          3.40   1050.0       0
2                          3.17   1185.0       0
3                          3.45   1480.0       0
4                          2.93    735.0       0
```




```
#cart regressor week 2
import pandas as pd
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.datasets import load_diabetes
import matplotlib.pyplot as plt

# Load the Diabetes dataset
diabetes = load_diabetes()
df = pd.DataFrame(data=diabetes.data, columns=diabetes.feature_names)
df['target'] = diabetes.target

# Print the first few rows of the dataset
print(df.head())

# Split the data into features and target
```

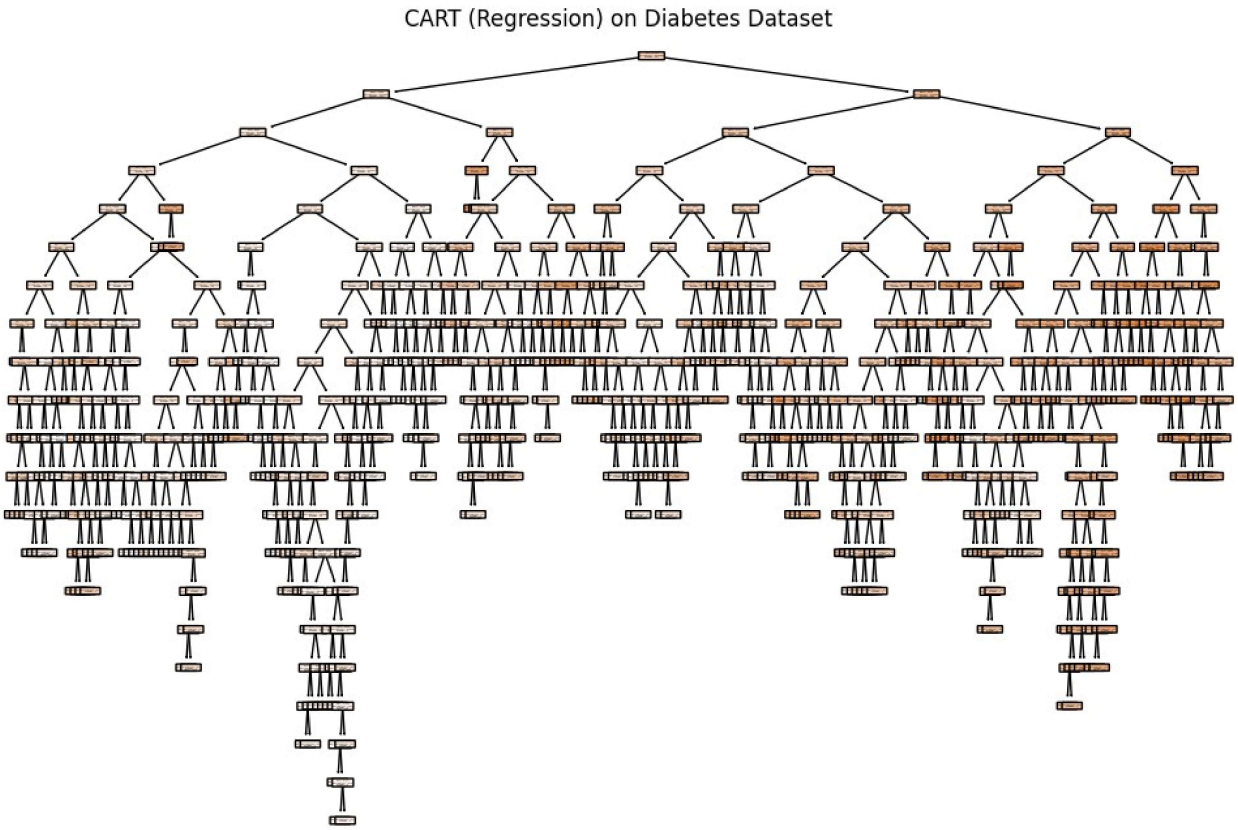
```
X = df.drop(columns=['target'])
y = df['target']

# Initialize the DecisionTreeRegressor with the CART algorithm (using
'mse' criterion)
model = DecisionTreeRegressor(random_state=42)
model.fit(X, y)

# Plot the decision tree
plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=diabetes.feature_names, filled=True,
rounded=True)
plt.title("CART (Regression) on Diabetes Dataset")
plt.show()
```

```
        age       sex       bmi        bp        s1        s2
s3  \
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -
0.043401
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163
0.074412
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -
0.032356
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -
0.036038
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596
0.008142

         s4        s5        s6  target
0 -0.002592  0.019907 -0.017646   151.0
1 -0.039493 -0.068332 -0.092204    75.0
2 -0.002592  0.002861 -0.025930   141.0
3  0.034309  0.022688 -0.009362   206.0
4 -0.002592 -0.031988 -0.046641   135.0
```




```
# simple Linear Regression week 3
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

df = pd.read_csv("C:/Users/shanm/Downloads/archive (2)/diabetes.csv")
x = df[['Insulin']]
y = df[['BMI']]

x train, x test, y train, y test = train_test_split(x, y,
test_size=0.3, random_state=42)

model = LinearRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

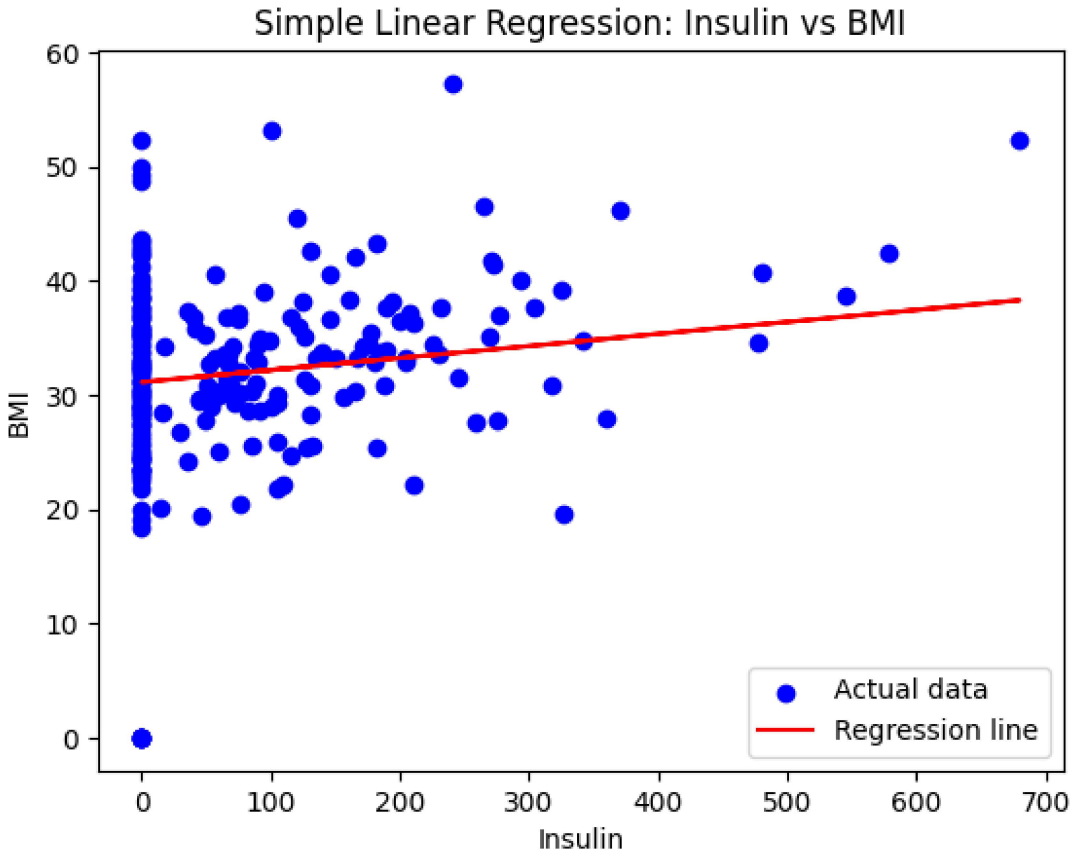
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')
print(f'R-squared: {r2}')
```

```
print(f'Coefficients: {model.coef_}')
print(f'Intercept: {model.intercept_}')

plt.scatter(x_test, y_test, color='blue', label='Actual data')
plt.plot(x_test, y_pred, color='red', label='Regression line')
plt.xlabel('Insulin')
plt.ylabel('BMI')
plt.title('Simple Linear Regression: Insulin vs BMI')
plt.legend()
plt.show()
```

```
Mean Squared Error: 67.23798721984811
R-squared: 0.05963378933341623
Coefficients: [[0.01053011]]
Intercept: [31.13492188]
```




```
#multiple linear regression week 3
import pandas as pd
from sklearn.model selection import train test split
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

df = pd.read_csv("C:/Users/shanm/Downloads/archive (2)/diabetes.csv")
x = df[['Insulin', 'Glucose', 'BMI', 'Age', 'BloodPressure']]
y = df[['BMI']]

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

x_train, x_test, y_train, y_test = train_test_split(x_scaled, y,
test_size=0.3, random_state=42)

model = LinearRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

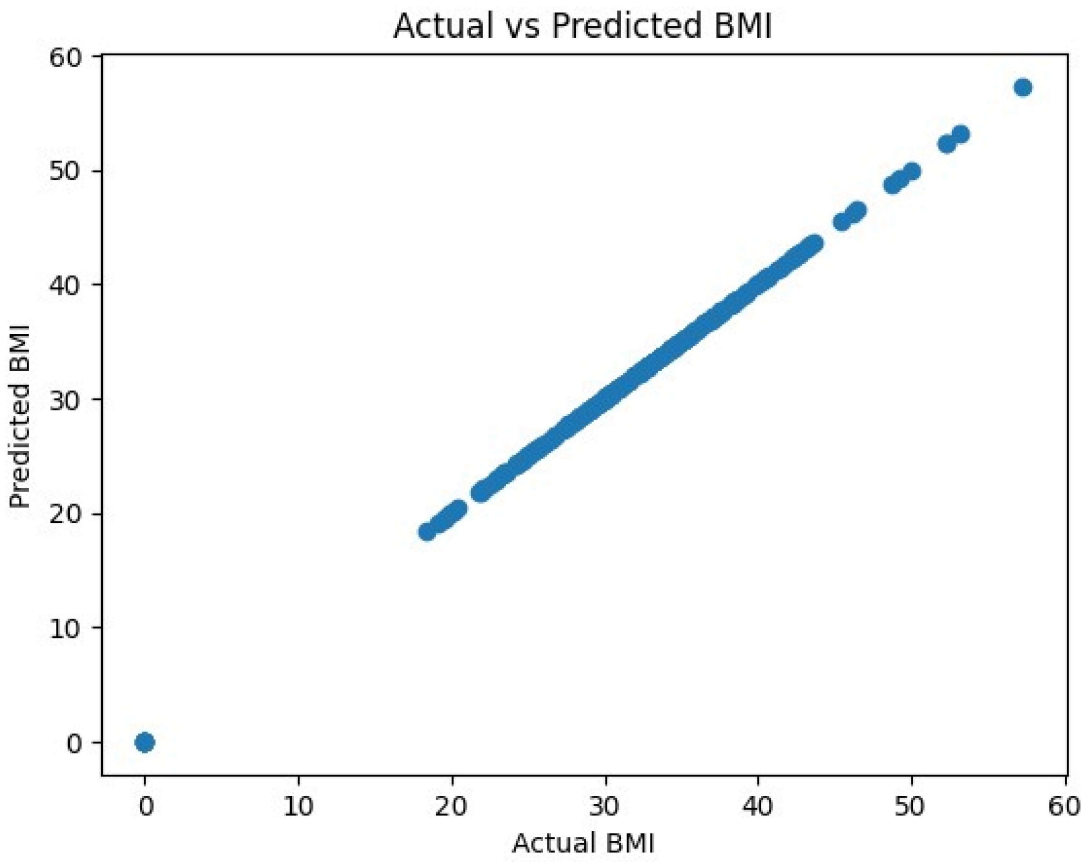
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

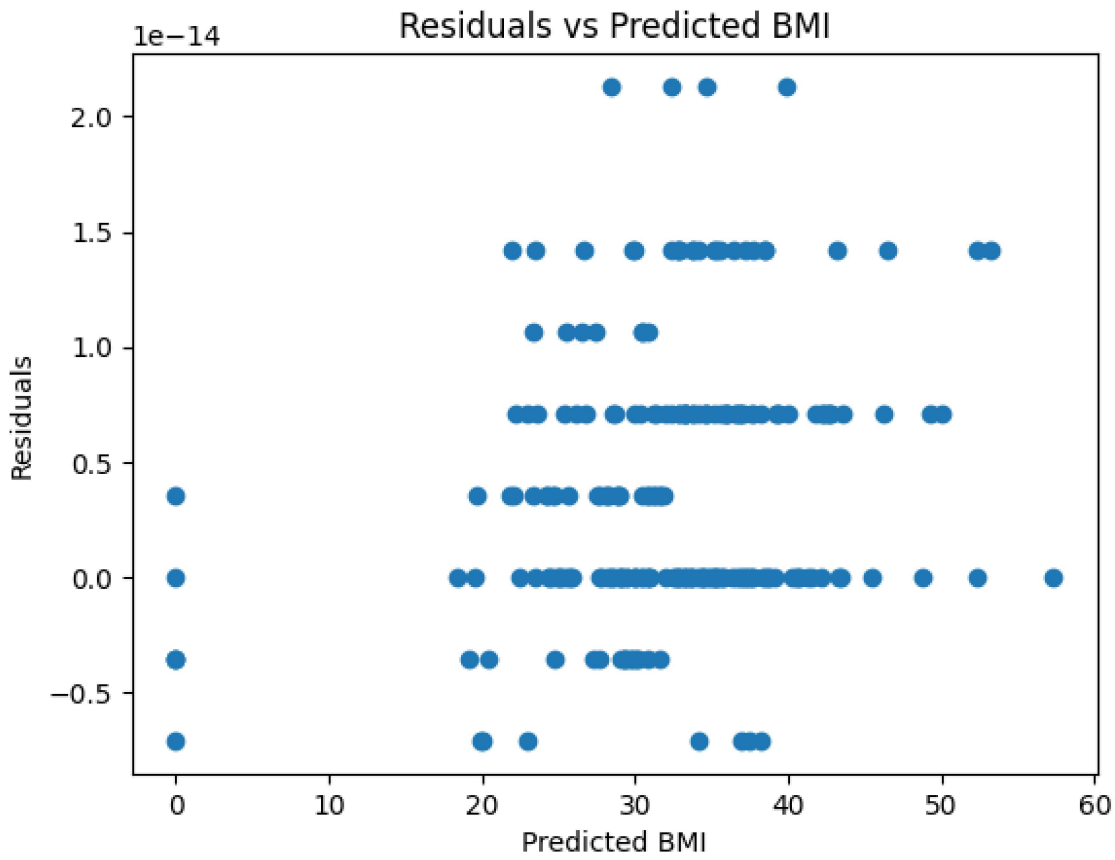
print(f'Mean Squared Error: {mse}')
print(f'R-squared: {r2}')
print(f'Coefficients: {model.coef_}')
print(f'Intercept: {model.intercept_}')

plt.scatter(y_test, y_pred)
plt.xlabel('Actual BMI')
plt.ylabel('Predicted BMI')
plt.title('Actual vs Predicted BMI')
plt.show()

residuals = y_test - y_pred
plt.scatter(y_pred, residuals)
plt.xlabel('Predicted BMI')
plt.ylabel('Residuals')
plt.title('Residuals vs Predicted BMI')
plt.show()
```

```
Mean Squared Error: 4.873862701001853e-29
R-squared: 1.0
Coefficients: [[ 1.04424161e-15 -3.99680289e-15  7.87902573e+00 -
2.38697950e-15
   2.47024623e-15]]
Intercept: [31.99257812]
```


Actual vs Predicted BMI
Predicted BMI
Actual BMI
0
10
20
30
40
50
60




```
#logistic Regression week 3
#Logistic Regression is used for classification tasks, you need to
ensure that the target variable (y) is categorical. If you intend to
classify the target variable (BMI), you may want to binarize or
categorize it.
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("C:/Users/shanm/Downloads/archive (2)/diabetes.csv")
x = df[['Insulin', 'Glucose', 'BMI', 'Age', 'BloodPressure']]

y = (df['BMI'] > 30).astype(int)

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

x_train, x_test, y_train, y_test = train_test_split(x_scaled, y,
```

```
test_size=0.3, random_state=42)

model = LogisticRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

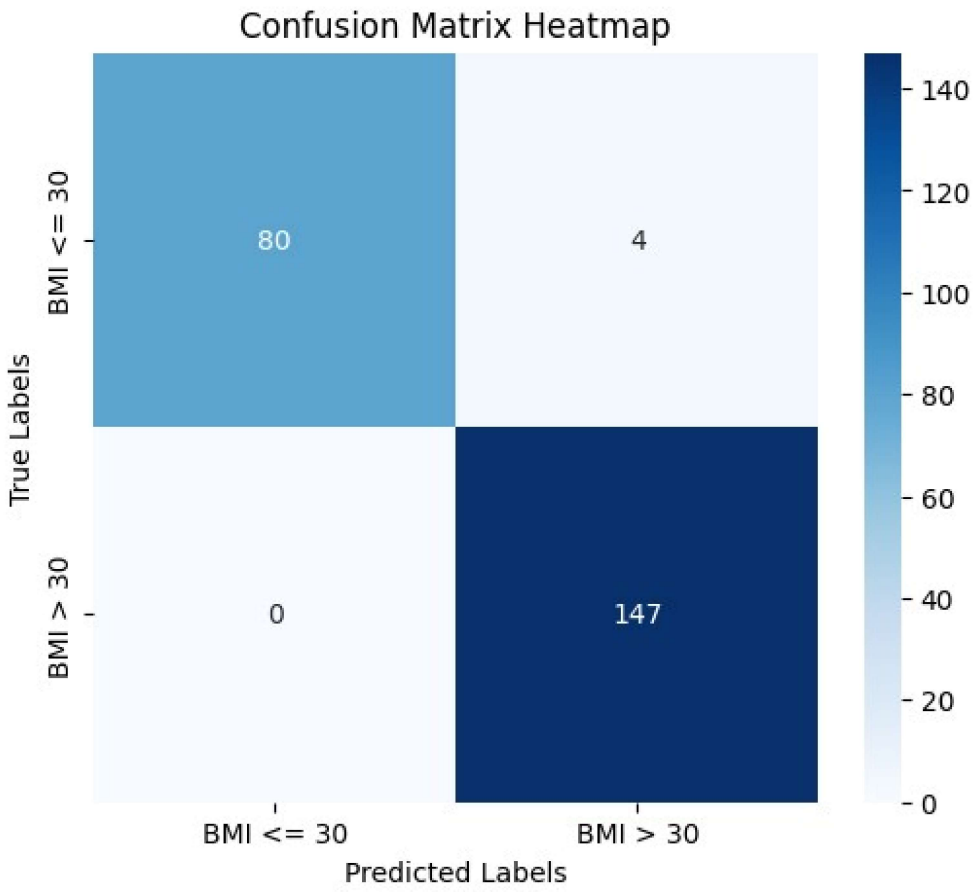
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['BMI
<= 30', 'BMI > 30'], yticklabels=['BMI <= 30', 'BMI > 30'])
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix Heatmap')
plt.show()
```

```
Accuracy: 98.27%
              precision    recall  f1-score   support

           0       1.00      0.95      0.98        84
           1       0.97      1.00      0.99       147

    accuracy                           0.98       231
   macro avg       0.99      0.98      0.98       231
weighted avg       0.98      0.98      0.98       231

[[ 80   4]
 [  0 147]]
```




```
#Linear SVM week 4 classifier
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score,
classification report
import seaborn as sns

data = load_breast_cancer()
X = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

model = SVC(kernel='linear')
model.fit(x_train, y_train)

y_pred = model.predict(x_test)
```

```python
print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
print(cm)

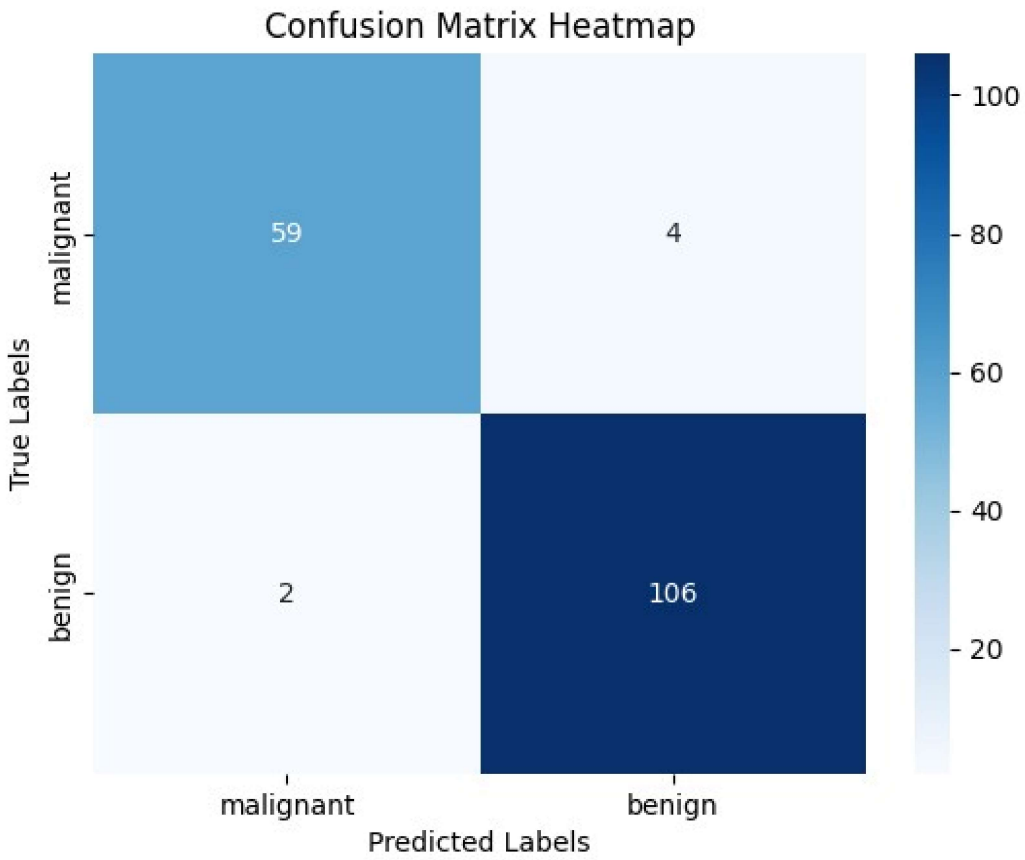
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=data.target_names, yticklabels=data.target_names)
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix Heatmap')
plt.show()
```

```
Accuracy: 96.49%
              precision    recall  f1-score   support

           0       0.97      0.94      0.95        63
           1       0.96      0.98      0.97       108

    accuracy                           0.96       171
   macro avg       0.97      0.96      0.96       171
weighted avg       0.96      0.96      0.96       171

[[ 59   4]
 [  2 106]]
```




```
#non linear svm regressor
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler

# Load the Diabetes dataset
data = load_diabetes()
X = data.data
y = data.target

# Split the data into training and test sets
x_train, x_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Standardize the data
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

```
# Create and train the model with tuned hyperparameters
model = SVR(kernel='rbf', C=100, epsilon=0.1)  # Using RBF kernel and
hyperparameters
model.fit(x_train_scaled, y_train)

# Make predictions
y_pred = model.predict(x_test_scaled)

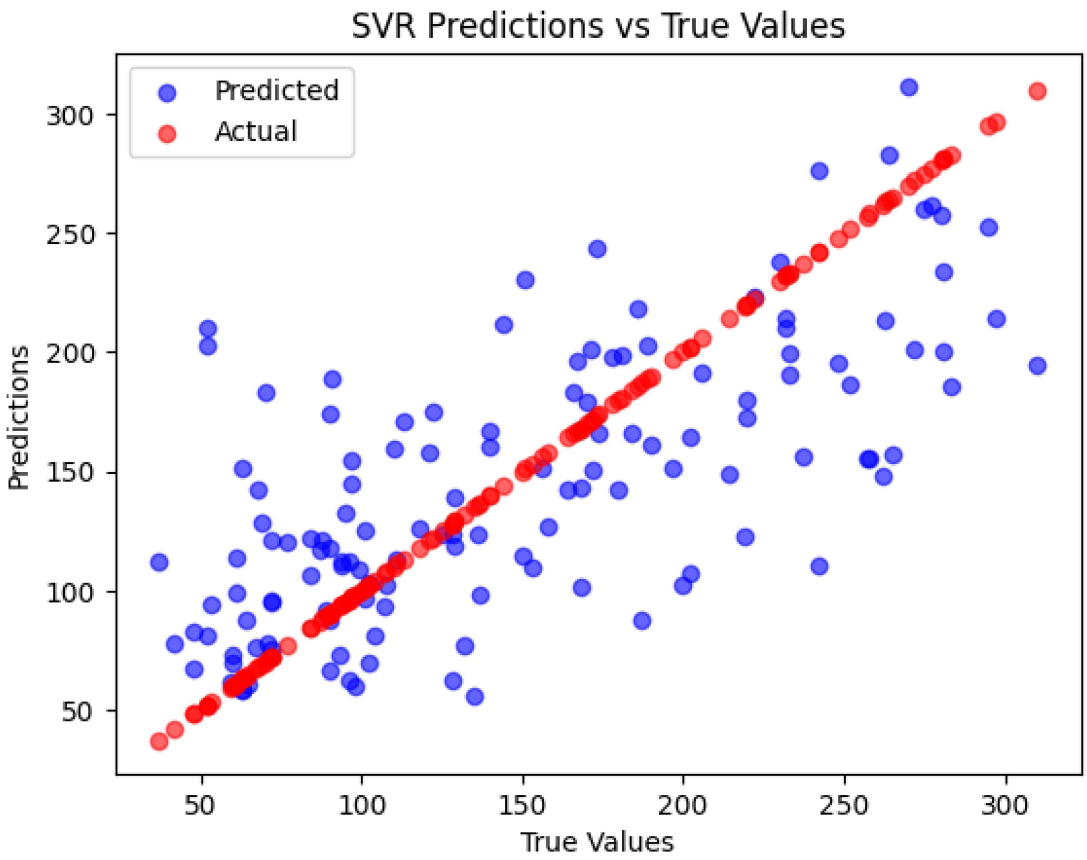
# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R^2 Score: {r2}")

# Plotting the results with different colors for predicted and actual
values
plt.scatter(y_test, y_pred, color='blue', label='Predicted',
alpha=0.6)
plt.scatter(y_test, y_test, color='red', label='Actual', alpha=0.6)

plt.xlabel('True Values')
plt.ylabel('Predictions')
plt.title('SVR Predictions vs True Values')
plt.legend()
plt.show()
```

```
Mean Squared Error: 2822.700660735967
R^2 Score: 0.47711379468482795
```




```
#NON-Linear SVM week 4 clssifier
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report
import matplotlib.pyplot as plt
import seaborn as sns

iris = load_iris()
x = iris.data
y = iris.target

x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, random_state=42)

# Train and predict with RBF kernel
model_rbf = SVC(kernel='rbf')
model_rbf.fit(x_train, y_train)
y_pred_rbf = model_rbf.predict(x_test)

# Train and predict with Polynomial kernel
```

```
model_poly = SVC(kernel='poly')
model_poly.fit(x_train, y_train)
y_pred_poly = model_poly.predict(x_test)

# Evaluate RBF kernel model
acs_rbf = accuracy_score(y_pred_rbf, y_test)
cm_rbf = confusion_matrix(y_pred_rbf, y_test)
cr_rbf = classification_report(y_pred_rbf, y_test)

# Evaluate Polynomial kernel model
acs_poly = accuracy_score(y_pred_poly, y_test)
cm_poly = confusion_matrix(y_pred_poly, y_test)
cr_poly = classification_report(y_pred_poly, y_test)

# Print results
print(f"RBF Kernel Accuracy: {acs_rbf * 100:.2f}%")
print("RBF Kernel Confusion Matrix:\n", cm_rbf)
print("RBF Kernel Classification Report:\n", cr_rbf)

print(f"Polynomial Kernel Accuracy: {acs_poly * 100:.2f}%")
print("Polynomial Kernel Confusion Matrix:\n", cm_poly)
print("Polynomial Kernel Classification Report:\n", cr_poly)

# Plot Confusion Matrices
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# RBF Kernel Confusion Matrix plot
sns.heatmap(cm_rbf, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names,
ax=axes[0])
axes[0].set_title('RBF Kernel Confusion Matrix')
axes[0].set_xlabel('Predicted Labels')
axes[0].set_ylabel('True Labels')

# Polynomial Kernel Confusion Matrix plot
sns.heatmap(cm_poly, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names,
ax=axes[1])
axes[1].set_title('Polynomial Kernel Confusion Matrix')
axes[1].set_xlabel('Predicted Labels')
axes[1].set_ylabel('True Labels')

plt.tight_layout()
plt.show()


RBF Kernel Accuracy: 100.00%
RBF Kernel Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
```

```
 [ 0  0 13]]
RBF Kernel Classification Report:
              precision    recall  f1-score   support

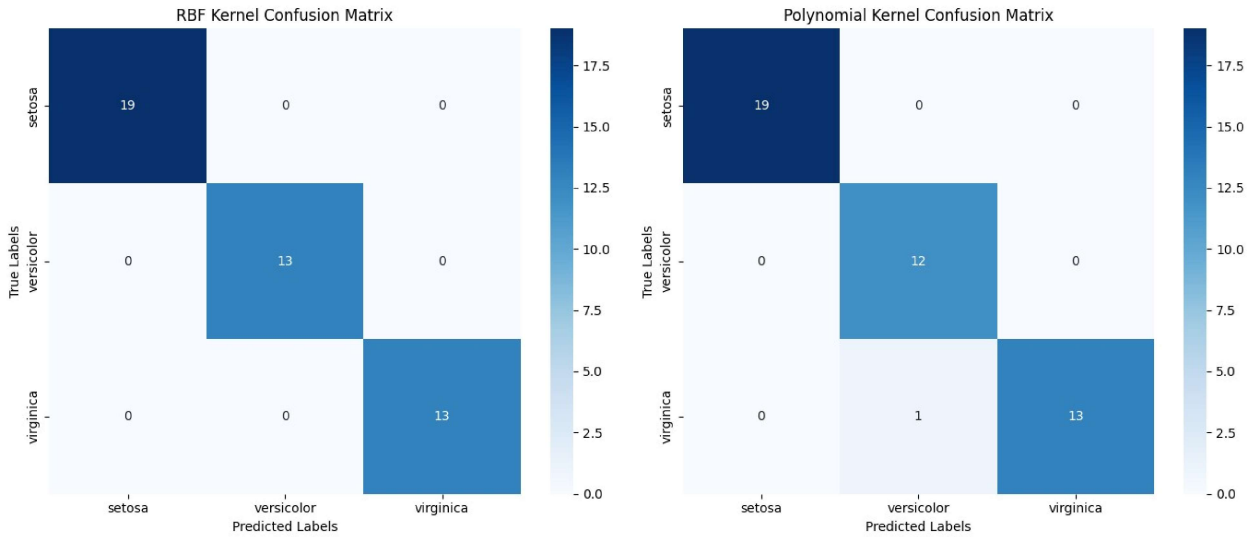
           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45

Polynomial Kernel Accuracy: 97.78%
Polynomial Kernel Confusion Matrix:
 [[19  0  0]
 [ 0 12  0]
 [ 0  1 13]]
Polynomial Kernel Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       0.92      1.00      0.96        12
           2       1.00      0.93      0.96        14

    accuracy                           0.98        45
   macro avg       0.97      0.98      0.97        45
weighted avg       0.98      0.98      0.98        45
```




```python
#KMEANS USING IRIS   week 5
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt
import seaborn as sns

iris = load_iris()
X = iris.data
y = iris.target

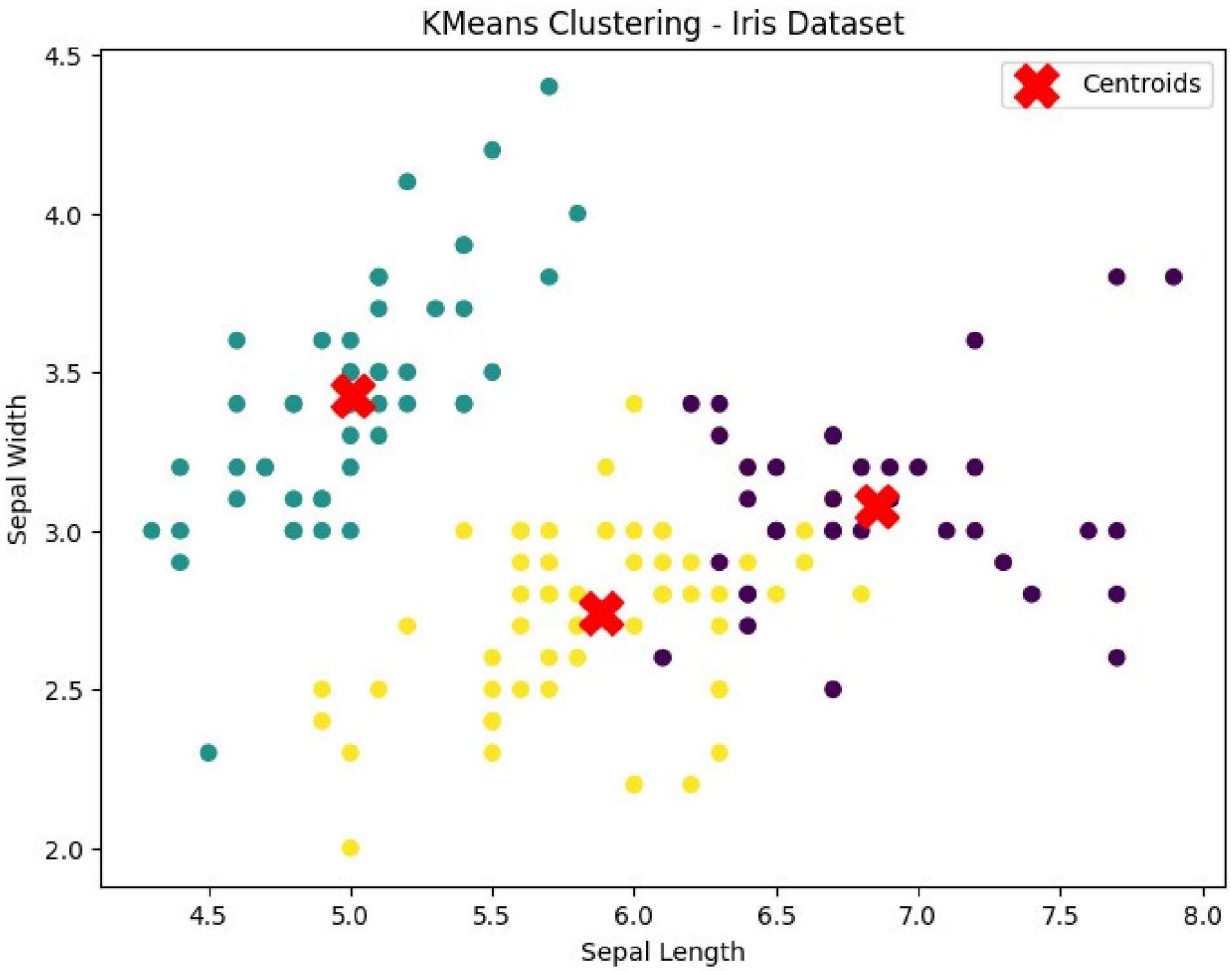

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

# Get the predicted cluster labels
y_pred = kmeans.predict(X)

# Plotting the clusters
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis', marker='o')

# Mark the centroids
centroids = kmeans.cluster_centers_
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='red',
marker='X', label='Centroids')

plt.title("KMeans Clustering - Iris Dataset")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.legend()
plt.show()
```




```
#K-Means using breast cancer  week 5
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import seaborn as sns
from sklearn.decomposition import PCA

df = pd.read_csv("C:/Users/shanm/Downloads/archive (1)/breast-
cancer.csv")

# Drop the 'diagnosis' column (target variable)
df = df.drop(['diagnosis'], axis=1)
# Convert categorical variables into dummy variables (one-hot
encoding)
# Dummy variables are used here to convert categorical features into
numerical representations
# This is necessary because machine learning algorithms like KMeans
require numerical input.
```

```
# For categorical columns, `get_dummies()` creates a new binary column
for each category.
# `drop_first=True` is used to avoid multicollinearity by removing one
category, as it can be inferred from the others.
df = pd.get_dummies(df, drop_first=True)

# Standardize the data (important for KMeans, as it is sensitive to
scale)
sc = StandardScaler()
df_scaled = sc.fit_transform(df)

# Elbow method to determine the optimal number of clusters
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i)
    kmeans.fit(df_scaled)
    wcss.append(kmeans.inertia_)

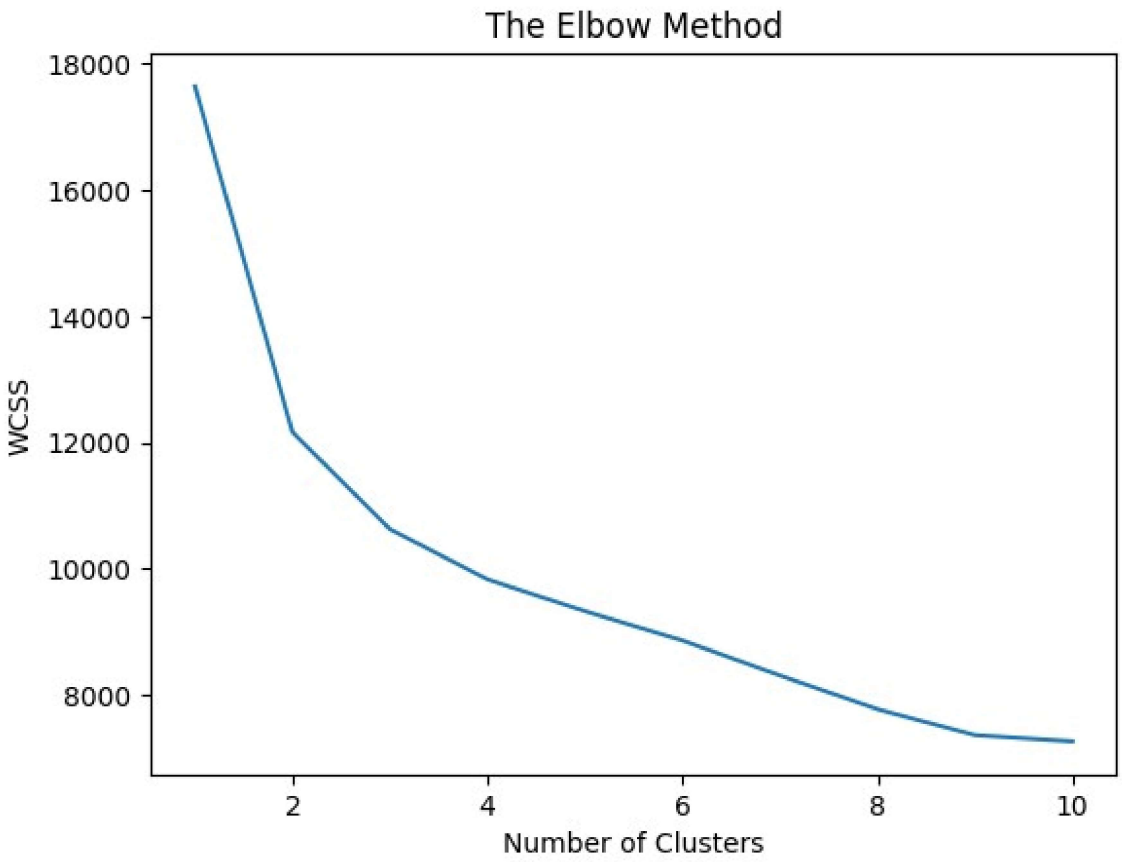
plt.plot(range(1, 11), wcss)
plt.title("The Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()

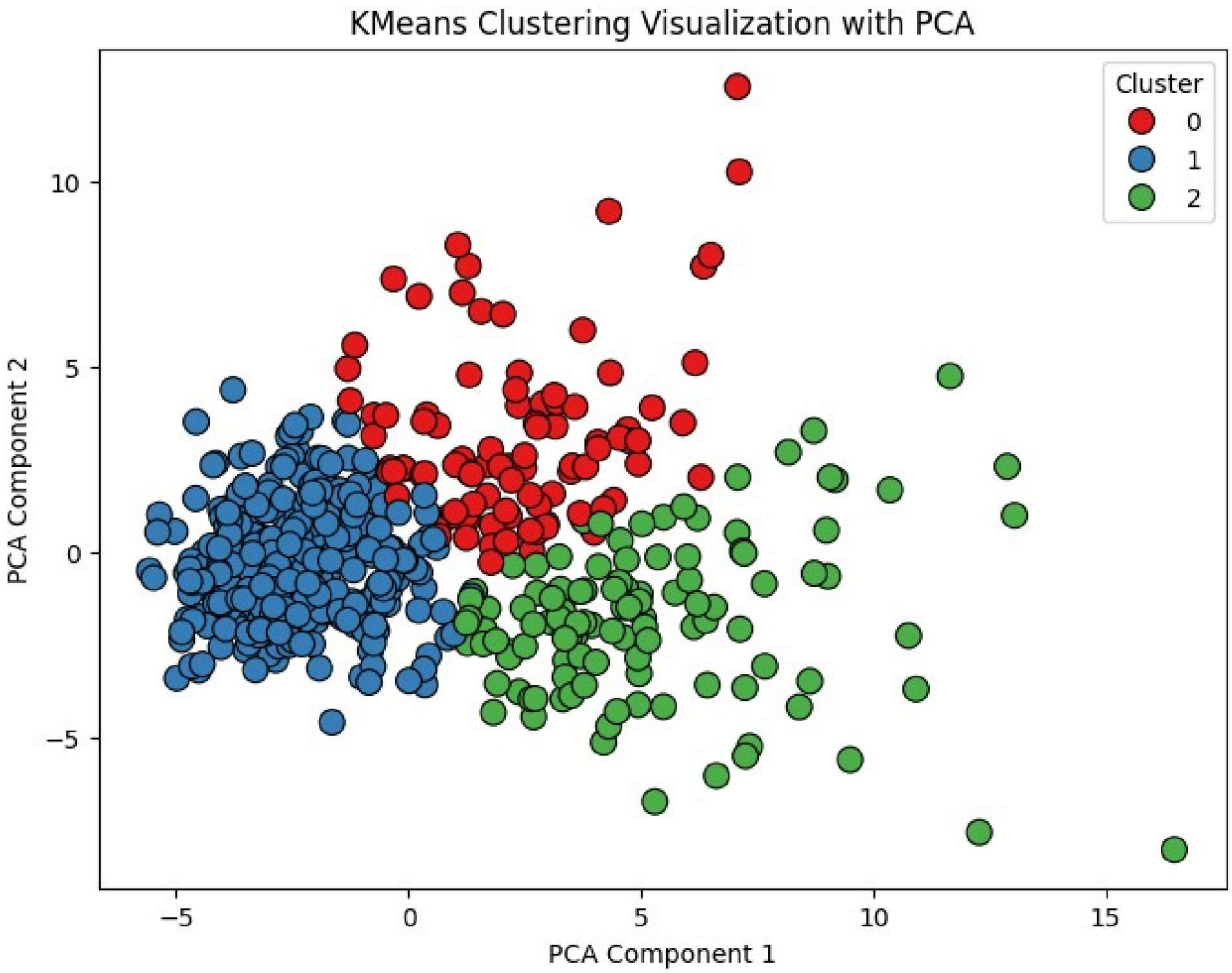
# Fit KMeans with the optimal number of clusters (3 in this case)
kmeans = KMeans(n_clusters=3)
df['cluster'] = kmeans.fit_predict(df_scaled)

# Apply PCA for dimensionality reduction (to 2D)
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)

# Create a DataFrame for the PCA results with the cluster labels
pca_df = pd.DataFrame(df_pca, columns=['PCA1', 'PCA2'])
pca_df['cluster'] = df['cluster']

# Plot the clusters in the PCA-reduced 2D space
plt.figure(figsize=(8,6))
sns.scatterplot(x='PCA1', y='PCA2', hue='cluster', palette='Set1',
data=pca_df, s=100, edgecolor='black')
plt.title("KMeans Clustering Visualization with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.legend(title='Cluster')
plt.show()
```


The Elbow Method
18000
16000
14000
12000
10000
8000
WCSS
2
4
6
8
10
Number of Clusters




```
# EM (Expectation-Maximization) algorithm vs the K-Means algorithm
week 6
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score

df = pd.read_csv("C:/Users/shanm/Downloads/archive (1)/breast-
cancer.csv")
# Drop the target variable (diagnosis) as we are performing
unsupervised learning
df = df.drop(['diagnosis'], axis=1)


scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
```

```
# Apply K-Means Clustering
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans_clusters = kmeans.fit_predict(df_scaled)

# Apply EM (Gaussian Mixture) Clustering
gmm = GaussianMixture(n_components=2, random_state=42)
gmm_clusters = gmm.fit_predict(df_scaled)

# Compare using Silhouette Score (higher is better)
kmeans_silhouette = silhouette_score(df_scaled, kmeans_clusters)
gmm_silhouette = silhouette_score(df_scaled, gmm_clusters)

print(f'K-Means Silhouette Score: {kmeans_silhouette}')
print(f'EM (GMM) Silhouette Score: {gmm_silhouette}')

# Use PCA to reduce the data to 2 dimensions for visualization
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)

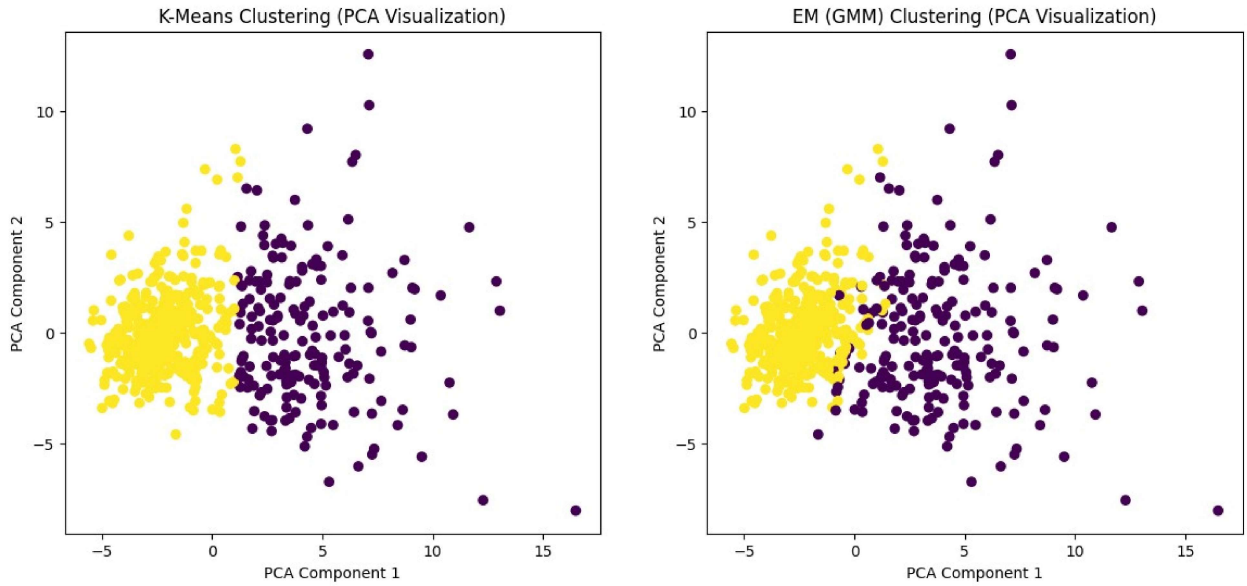
# Visualize the clusters formed by K-Means and EM (GMM) using PCA
plt.figure(figsize=(14, 6))

# K-Means visualization
plt.subplot(1, 2, 1)
plt.scatter(df_pca[:, 0], df_pca[:, 1], c=kmeans_clusters,
cmap='viridis')
plt.title("K-Means Clustering (PCA Visualization)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")

# EM (GMM) visualization
plt.subplot(1, 2, 2)
plt.scatter(df_pca[:, 0], df_pca[:, 1], c=gmm_clusters,
cmap='viridis')
plt.title("EM (GMM) Clustering (PCA Visualization)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")

plt.show()
```

```
K-Means Silhouette Score: 0.3382381691599185
EM (GMM) Silhouette Score: 0.30790772029801805
```




```
#Model Training: Holdout, K-Fold cross validation and Bootstrap
Sampling   week 7
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier,
RandomForestRegressor
from sklearn.metrics import accuracy_score, mean_squared_error,
r2_score
from sklearn.utils import resample

iris = load_iris()
x = iris.data
y = iris.target

# Holdout Method for Classification
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, random_state=42)

# RandomForestClassifier for classification task
clf = RandomForestClassifier()
clf.fit(x_train, y_train)
y_pred_class = clf.predict(x_test)
clf_accuracy = accuracy_score(y_test, y_pred_class)
print(f'RandomForestClassifier Holdout Accuracy: {clf_accuracy}')

# Cross-Validation for Classification
cvs_class = cross_val_score(clf, x, y, cv=5)
print(f'RandomForestClassifier Cross-Validation Accuracy (5 folds):
{cvs_class.mean()}')
```

```
# Bootstrapping for Classification
bootstrap_accuracies_class = []
for i in range(10):
    xr, yr = resample(x, y, replace=True, random_state=i)
    clf.fit(xr, yr)
    y_pred_class = clf.predict(x_test)
    clf_accuracy = accuracy_score(y_test, y_pred_class)
    bootstrap_accuracies_class.append(clf_accuracy)
    print(f'RandomForestClassifier Bootstrapping Accuracy (iteration
{i+1}): {clf_accuracy}')

# Plotting Bootstrapping Accuracies for Classification
plt.figure(figsize=(10, 5))
plt.plot(range(1, 11), bootstrap_accuracies_class, marker='o',
color='b', label='Bootstrap Accuracy')
plt.title('RandomForestClassifier Bootstrapping Accuracy')
plt.xlabel('Bootstrap Iteration')
plt.ylabel('Accuracy')
plt.xticks(range(1, 11))
plt.legend()
plt.grid(True)
plt.show()

# RandomForestRegressor for regression task (predict petal length)
y_reg = iris.data[:, 2]  # Petal Length as a continuous variable

# Holdout Method for Regression
x_train, x_test, y_train_reg, y_test_reg = train_test_split(x, y_reg,
test_size=0.3, random_state=42)

# RandomForestRegressor for regression task
reg = RandomForestRegressor()
reg.fit(x_train, y_train_reg)
y_pred_reg = reg.predict(x_test)

# Regression Metrics
mse = mean_squared_error(y_test_reg, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
print(f'RandomForestRegressor Holdout Mean Squared Error: {mse}')
print(f'RandomForestRegressor Holdout R^2 Score: {r2}')

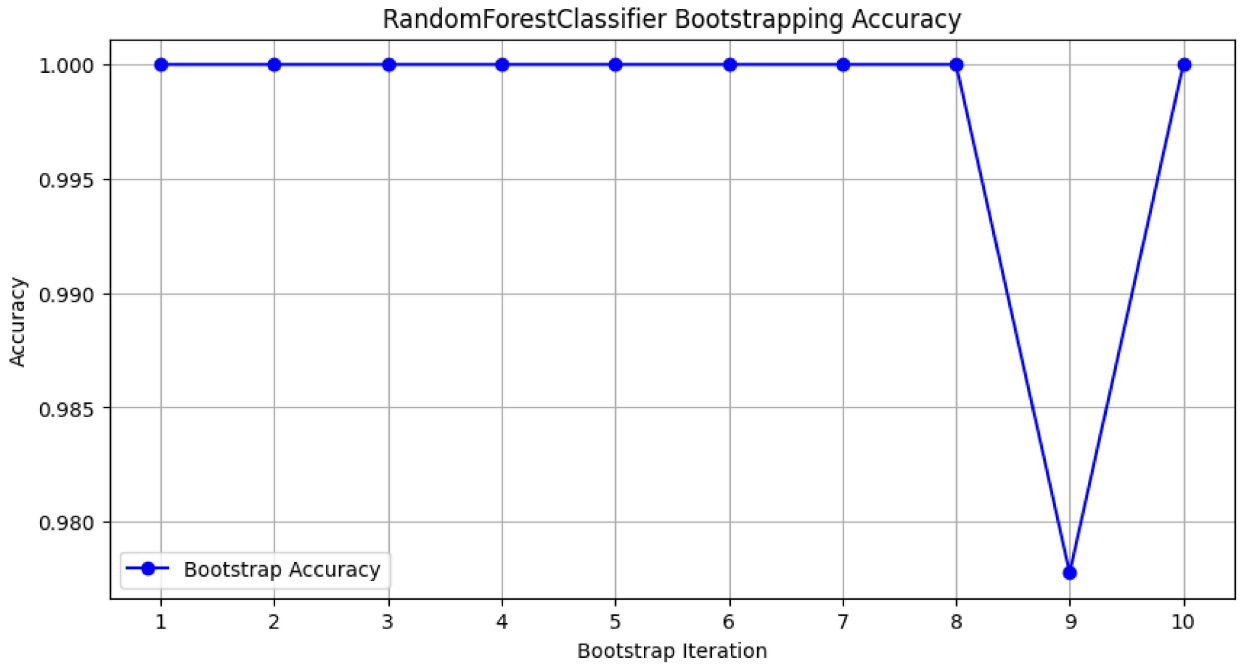
# Cross-Validation for Regression
cvs_reg = cross_val_score(reg, x, y_reg, cv=5)
print(f'RandomForestRegressor Cross-Validation R^2 Score (5 folds):
{cvs_reg.mean()}')

# Bootstrapping for Regression
bootstrap_mse_reg = []
for i in range(10):
```

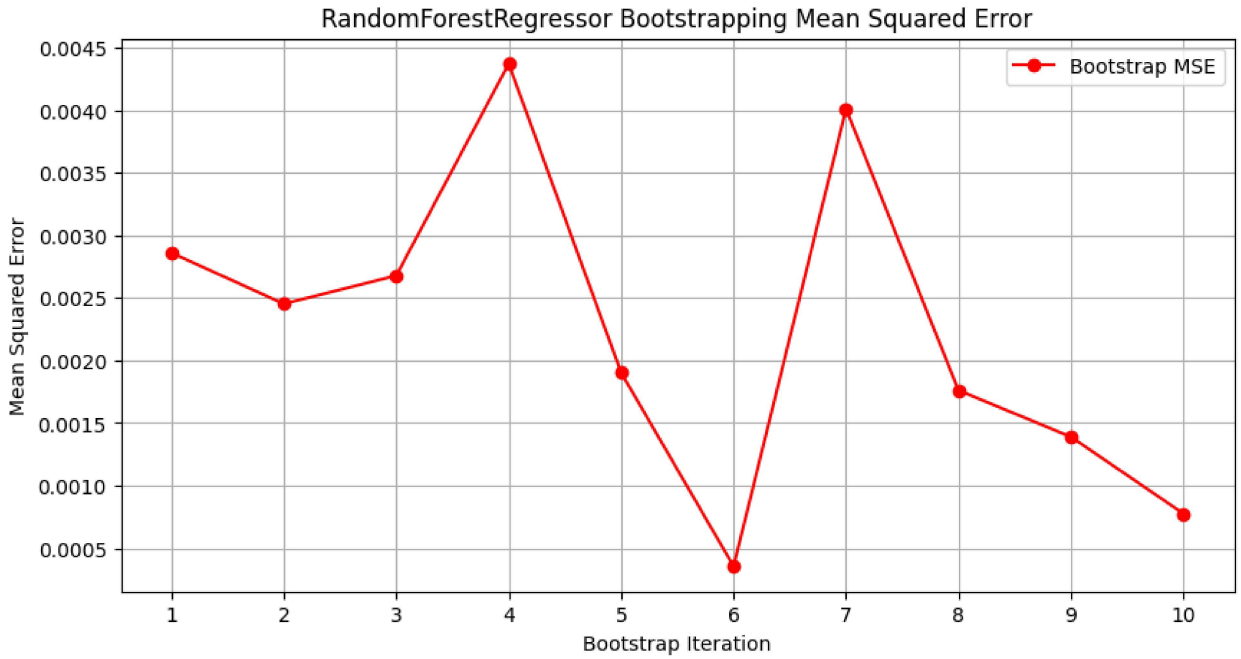
```
    xr, yr = resample(x, y_reg, replace=True, random_state=i)
    reg.fit(xr, yr)
    y_pred_reg = reg.predict(x_test)
    mse_reg = mean_squared_error(y_test_reg, y_pred_reg)
    bootstrap_mse_reg.append(mse_reg)
    print(f'RandomForestRegressor Bootstrapping Mean Squared Error
(iteration {i+1}): {mse_reg}')

# Plotting Bootstrapping MSE for Regression
plt.figure(figsize=(10, 5))
plt.plot(range(1, 11), bootstrap_mse_reg, marker='o', color='r',
label='Bootstrap MSE')
plt.title('RandomForestRegressor Bootstrapping Mean Squared Error')
plt.xlabel('Bootstrap Iteration')
plt.ylabel('Mean Squared Error')
plt.xticks(range(1, 11))
plt.legend()
plt.grid(True)
plt.show()
```

```
RandomForestClassifier Holdout Accuracy: 1.0
RandomForestClassifier Cross-Validation Accuracy (5 folds): 0.96
RandomForestClassifier Bootstrapping Accuracy (iteration 1): 1.0
RandomForestClassifier Bootstrapping Accuracy (iteration 2): 1.0
RandomForestClassifier Bootstrapping Accuracy (iteration 3): 1.0
RandomForestClassifier Bootstrapping Accuracy (iteration 4): 1.0
RandomForestClassifier Bootstrapping Accuracy (iteration 5): 1.0
RandomForestClassifier Bootstrapping Accuracy (iteration 6): 1.0
RandomForestClassifier Bootstrapping Accuracy (iteration 7): 1.0
RandomForestClassifier Bootstrapping Accuracy (iteration 8): 1.0
RandomForestClassifier Bootstrapping Accuracy (iteration 9):
0.9777777777777777
RandomForestClassifier Bootstrapping Accuracy (iteration 10): 1.0
```




```
RandomForestRegressor Holdout Mean Squared Error:
0.0027637555555555526
RandomForestRegressor Holdout R^2 Score: 0.9991803761133876
RandomForestRegressor Cross-Validation R^2 Score (5 folds):
0.9297855164638369
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 1):
0.0028612888888888545
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 2):
0.0024534666666668017
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 3):
0.0026784888888888927
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 4):
0.004373422222222198
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 5):
0.0019026222222221477
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 6):
0.0003540444444444651
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 7):
0.004011799999999872
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 8):
0.0017614888888888044
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 9):
0.0013892666666667235
RandomForestRegressor Bootstrapping Mean Squared Error (iteration 10):
0.0007757333333333648
```




```
#WEEK8

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier,
RandomForestRegressor
from sklearn.metrics import accuracy_score, classification_report,
mean_squared_error, r2_score, confusion_matrix
from sklearn.cluster import KMeans
from sklearn.preprocessing import LabelEncoder
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score

# Step 1: Load the Dataset
df = pd.read_csv("C:/Users/shanm/Downloads/archive
(5)/Dataset_spine.csv")

# Drop the extra column 'Unnamed: 13'
df = df.drop(columns=['Unnamed: 13'])

# Step 2: Encode the target column 'Class_att' to binary values
(Abnormal -> 1, Normal -> 0) for classification
label_encoder = LabelEncoder()
df['Class_att'] = label_encoder.fit_transform(df['Class_att'])

# Step 3: Split into features (X) and target (y)
X = df.drop('Class_att', axis=1)  # Features
```

```
y = df['Class_att']  # Target for classification

# Step 4: Supervised Learning - Classification (Random Forest
Classifier)
# Split the data into training and testing sets for classification
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Train a Random Forest Classifier
clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)

# Make predictions
y_pred_class = clf.predict(X_test)

# Evaluate the classification model
print("Classification Model Performance:")
print("Accuracy Score:", accuracy_score(y_test, y_pred_class))
print("Classification Report:")
print(classification_report(y_test, y_pred_class))

# Step 5: Confusion Matrix for Classification
cm = confusion_matrix(y_test, y_pred_class)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=['Normal', 'Abnormal'], yticklabels=['Normal',
'Abnormal'])
plt.title("Confusion Matrix - Classification")
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# Step 6: Supervised Learning - Regression (Random Forest Regressor)
# Ensure y is continuous for regression; here using 'Col1' as an
example feature for regression
y_reg = df['Col1']  # Let's assume we want to predict 'Col1' (or any
other continuous feature)

# Split the data into training and testing sets for regression
X_train, X_test, y_train_reg, y_test_reg = train_test_split(X, y_reg,
test_size=0.3, random_state=42)

# Train a Random Forest Regressor
reg = RandomForestRegressor(random_state=42)
reg.fit(X_train, y_train_reg)

# Make predictions
y_pred_reg = reg.predict(X_test)

# Evaluate the regression model
```

```python
print("Regression Model Performance:")
mse = mean_squared_error(y_test_reg, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
print(f"Mean Squared Error: {mse}")
print(f"R^2 Score: {r2}")

# Step 7: Unsupervised Learning - Clustering (KMeans)
# Perform KMeans clustering
kmeans = KMeans(n_clusters=3, random_state=42)
y_kmeans = kmeans.fit_predict(X)

# Evaluate clustering using Silhouette Score
sil_score = silhouette_score(X, y_kmeans)
print("Clustering - Silhouette Score:", sil_score)

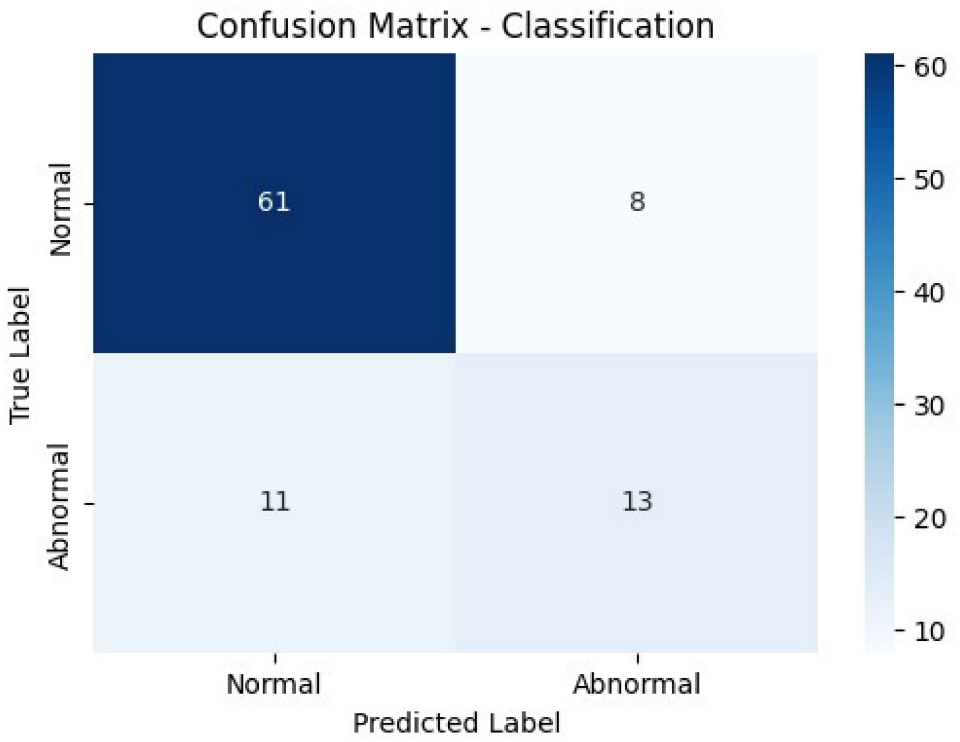
# Step 8: Evaluate Clustering Quality with Purity
def purity_score(y_true, y_pred):
    contingency_matrix = confusion_matrix(y_true, y_pred)
    purity = np.sum(np.amax(contingency_matrix, axis=0)) /
np.sum(contingency_matrix)
    return purity

# Calculate Purity Score
purity = purity_score(y, y_kmeans)
print("Clustering - Purity Score:", purity)
```

```
Classification Model Performance:
Accuracy Score: 0.7956989247311828
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.88      0.87        69
           1       0.62      0.54      0.58        24

    accuracy                           0.80        93
   macro avg       0.73      0.71      0.72        93
weighted avg       0.79      0.80      0.79        93
```




```
Regression Model Performance:
Mean Squared Error: 2.0931933999632206
R^2 Score: 0.99258718525602
Clustering - Silhouette Score: 0.3445780398072005
Clustering - Purity Score: 0.7032258064516129
```

```
#Random forest classifier week 9
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import seaborn as sns

# Load the iris dataset
iris = load_iris()
x = iris.data
y = iris.target

# Train-test split
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, random_state=42)

# RandomForestClassifier model
```

```
model = RandomForestClassifier()
model.fit(x_train, y_train)

# Make predictions
y_pred = model.predict(x_test)

# Evaluate the model
print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

# Plotting Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.show()

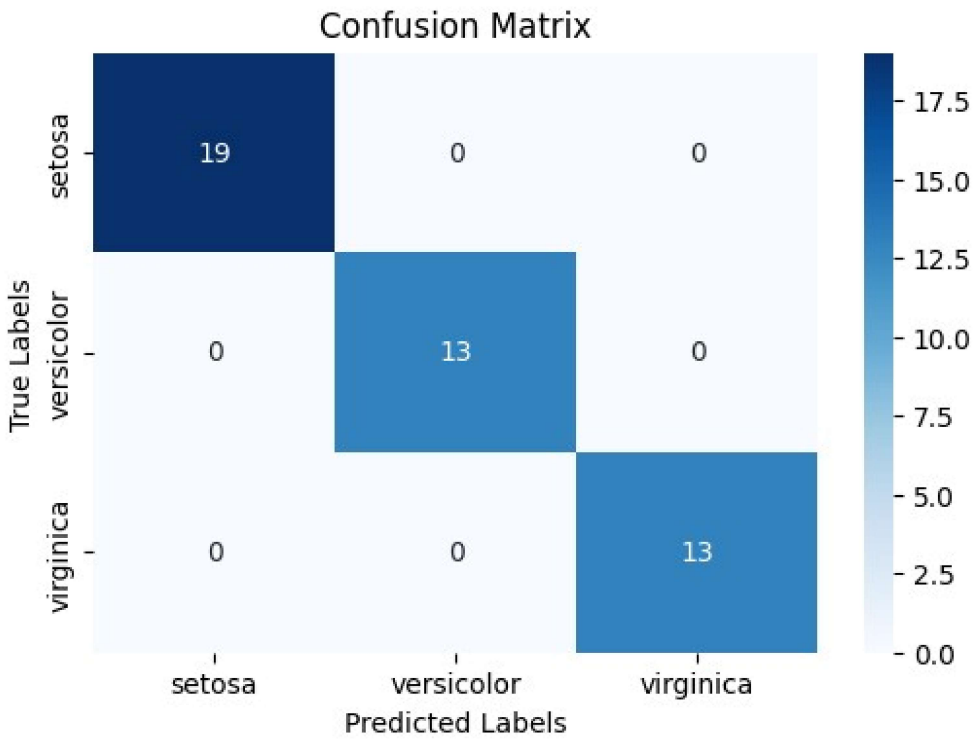
# Plotting Feature Importances
feature_importances = model.feature_importances_
plt.figure(figsize=(8, 6))
plt.barh(iris.feature_names, feature_importances, color='green')
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.show()
```

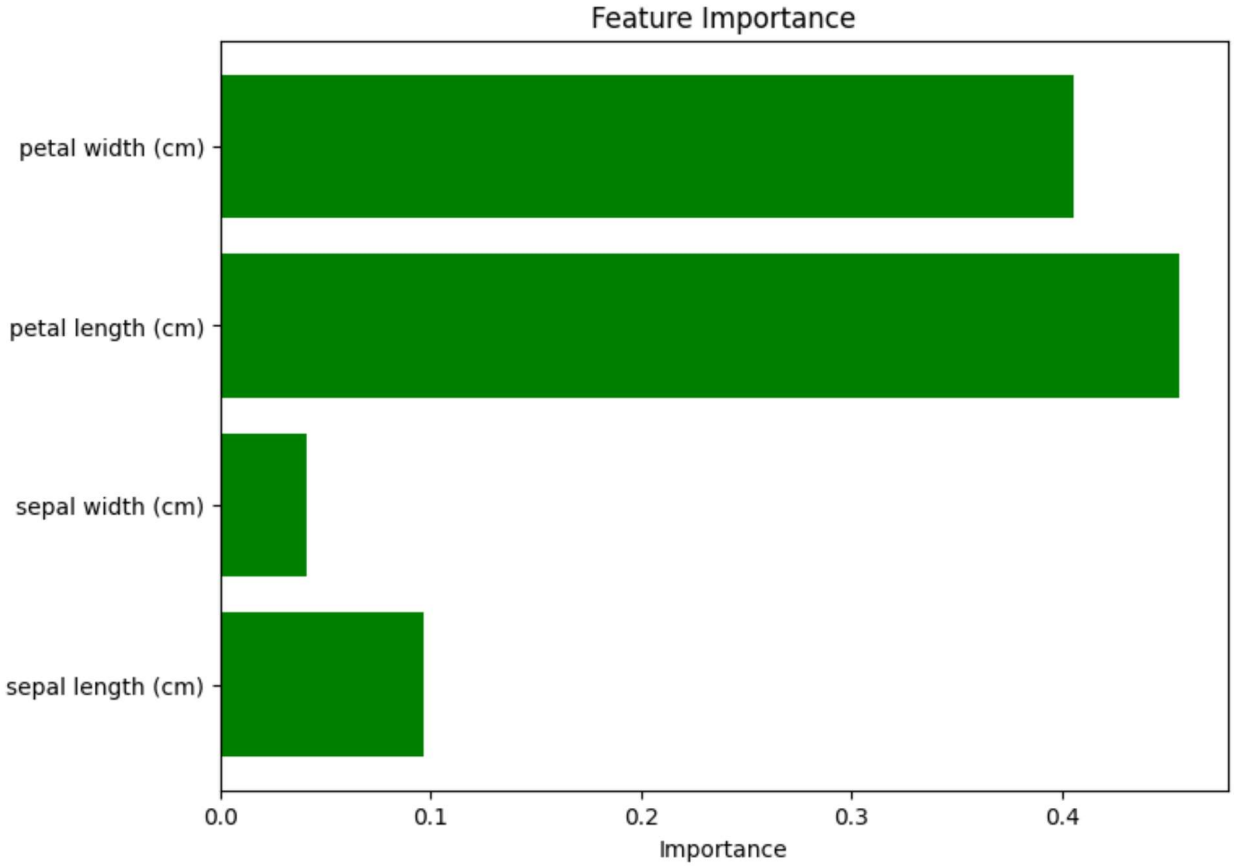
```
Accuracy: 100.00%
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45

[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```


Confusion Matrix
True Labels
setosa
versicolor
virginica
19
0
0
0
13
0
0
0
13
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
setosa
versicolor
virginica
Predicted Labels


Feature Importance
petal width (cm)
petal length (cm)
sepal width (cm)
sepal length (cm)
0.0
0.1
0.2
0.3
0.4
Importance

```python
#Random forest regressor  week 9
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Load the Diabetes dataset
diabetes = load diabetes()
X = diabetes.data
y = diabetes.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Initialize and train the RandomForestRegressor model
model = RandomForestRegressor()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

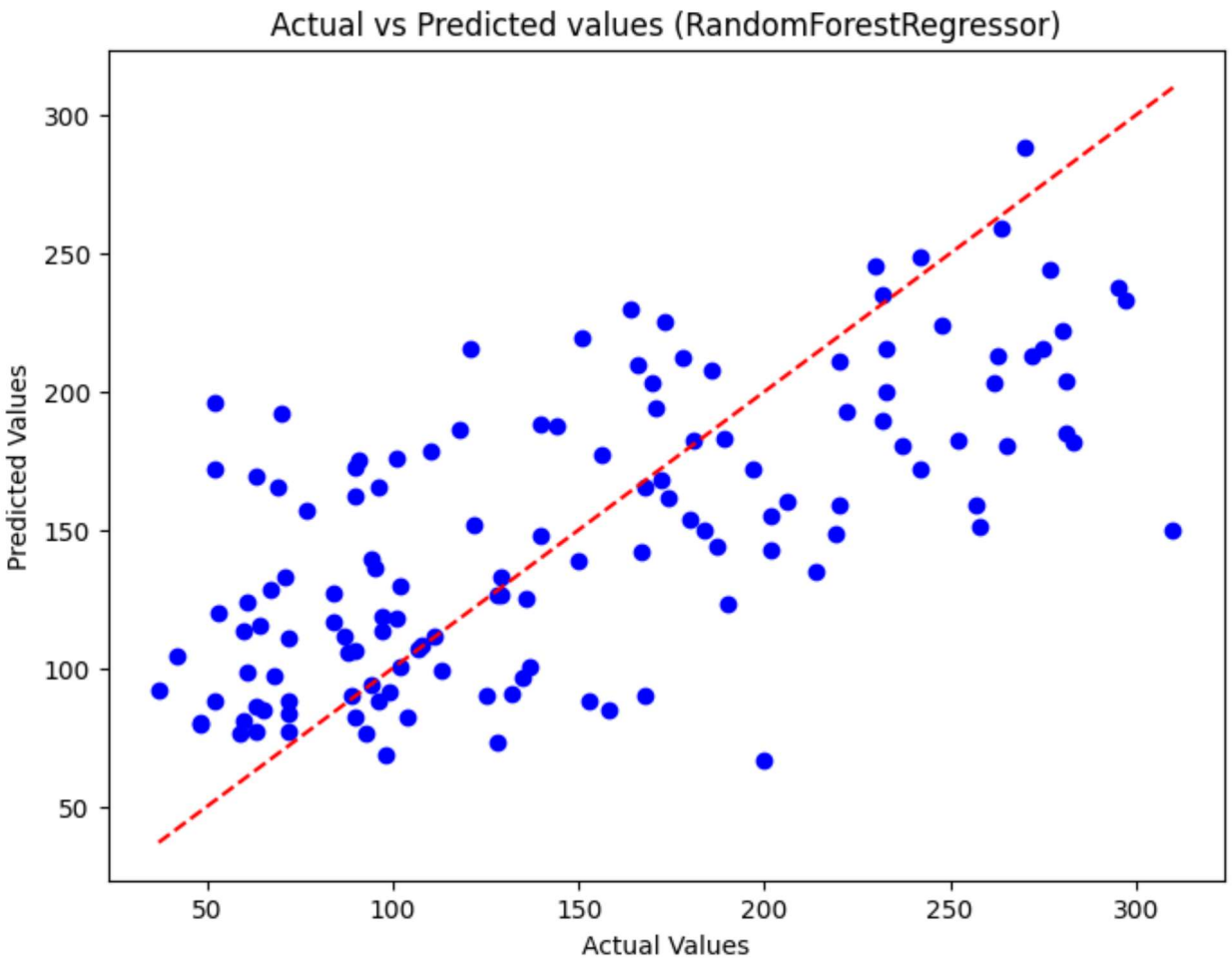
# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

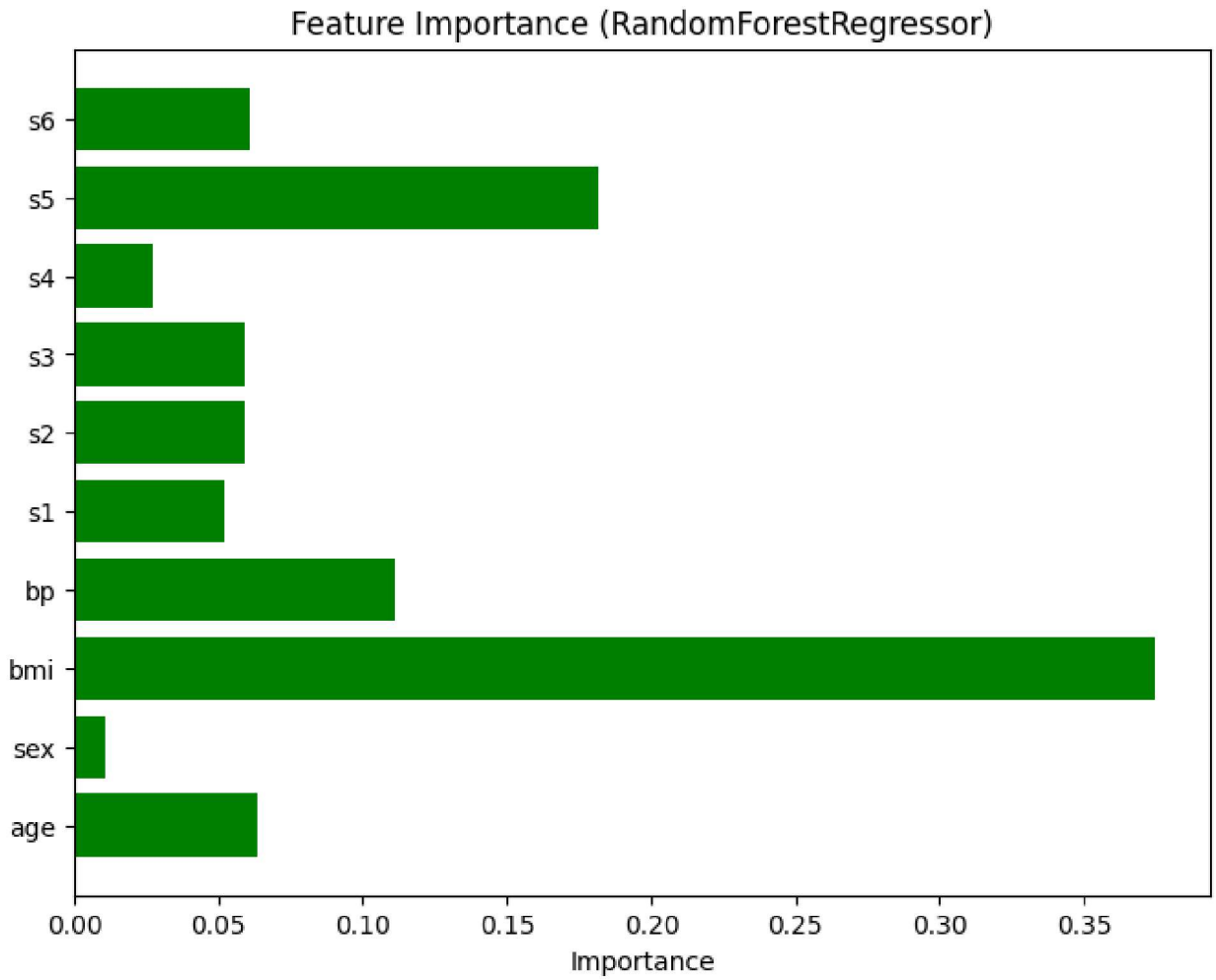
print(f"Mean Squared Error: {mse}")
print(f"R^2 Score: {r2}")

# Plot Predicted vs Actual values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red', linestyle='--')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Actual vs Predicted values (RandomForestRegressor)')
plt.show()

# Plot Feature Importances
feature_importances = model.feature_importances_
plt.figure(figsize=(8, 6))
plt.barh(diabetes.feature_names, feature_importances, color='green')
plt.xlabel('Importance')
plt.title('Feature Importance (RandomForestRegressor)')
plt.show()
```

```
Mean Squared Error: 2963.6213060150376
R^2 Score: 0.45100919830111985
```






```
#Adaboost week 10
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Load the Wine dataset
wine = load_wine()
X = wine.data
y = wine.target

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

```python
# AdaBoost with a decision tree classifier as the base learner
(Decision stump)
base_learner = DecisionTreeClassifier(max_depth=1)  # Decision stump
(shallow tree)
model = AdaBoostClassifier(base_learner, n_estimators=50,
random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict on the test data
y_pred = model.predict(X_test)

# Evaluate the model
print("AdaBoost Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Plot Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
plt.colorbar()
plt.xlabel('Predicted label')
plt.ylabel('True label')
plt.xticks(np.arange(len(np.unique(y))), np.unique(y))
plt.yticks(np.arange(len(np.unique(y))), np.unique(y))
plt.show()

# Plot Feature Importance
feature_importances = model.feature_importances_
plt.figure(figsize=(8, 6))
plt.barh(wine.feature_names, feature_importances, color='green')
plt.xlabel('Importance')
plt.title('Feature Importance (AdaBoost Classifier)')
plt.show()
```

```
C:\Users\shanm\AppData\Roaming\Python\Python311\site-packages\sklearn\
ensemble\_weight_boosting.py:527: FutureWarning: The SAMME.R algorithm
(the default) is deprecated and will be removed in 1.6. Use the SAMME
algorithm to circumvent this warning.
  warnings.warn(

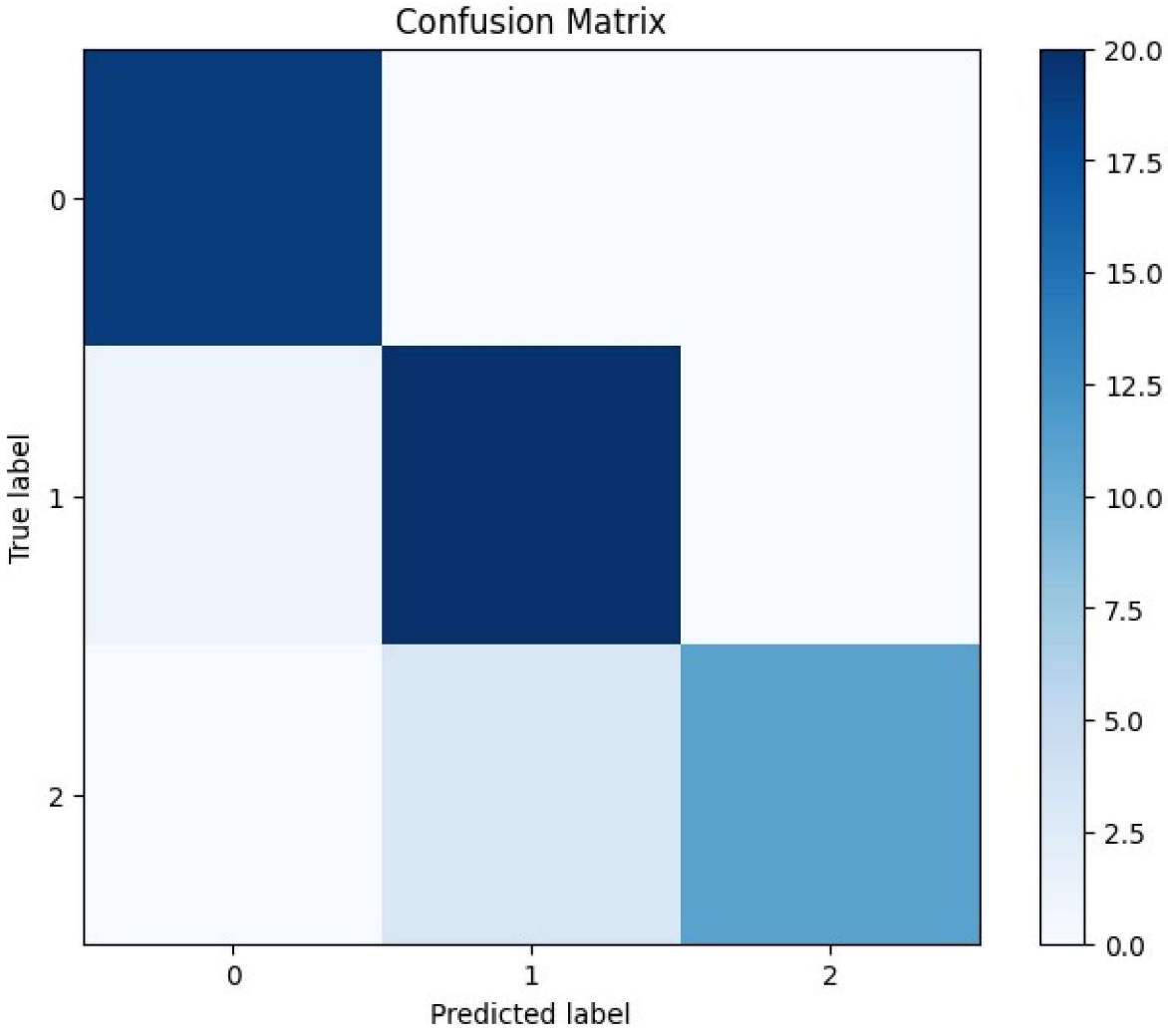
AdaBoost Accuracy: 0.9259259259259259
Classification Report:
              precision    recall  f1-score   support
```

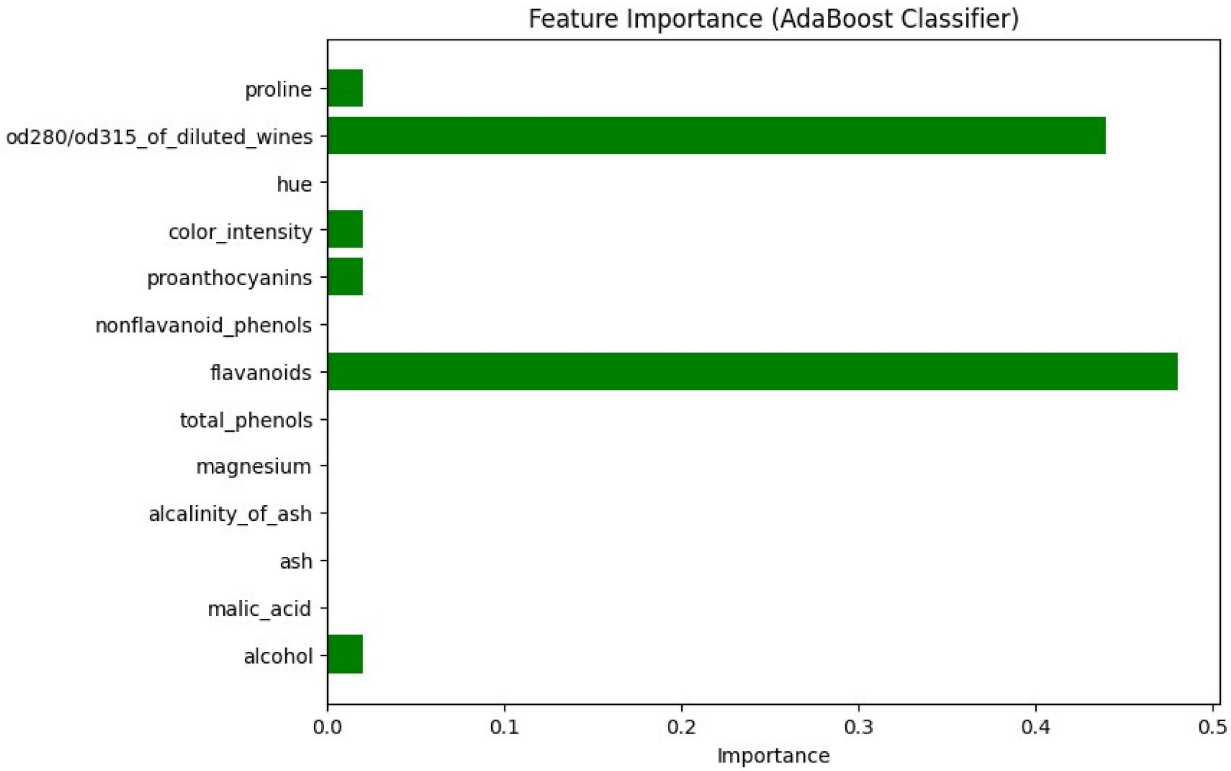
```
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        19
           1       0.87      0.95      0.91        21
           2       1.00      0.79      0.88        14

    accuracy                           0.93        54
   macro avg       0.94      0.91      0.92        54
weighted avg       0.93      0.93      0.92        54

Confusion Matrix:
 [[19  0  0]
 [ 1 20  0]
 [ 0  3 11]]
```






```
#XGBoost as classifier
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

wine = load_wine()
X = wine.data
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

model = XGBClassifier(n_estimators=50, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("XGBoost Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

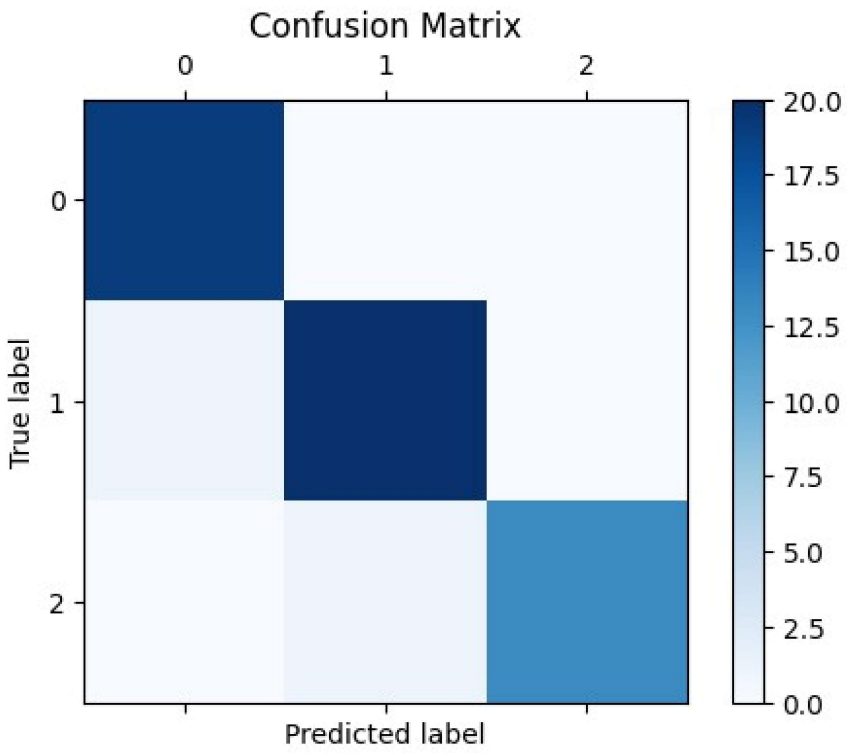
```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
plt.matshow(cm, cmap=plt.cm.Blues, fignum=1)
plt.title("Confusion Matrix")
plt.colorbar()
plt.xlabel('Predicted label')
plt.ylabel('True label')
plt.xticks(np.arange(len(np.unique(y))), np.unique(y))
plt.yticks(np.arange(len(np.unique(y))), np.unique(y))
plt.show()
```

```
XGBoost Accuracy: 0.9629629629629629
Classification Report:
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        19
           1       0.95      0.95      0.95        21
           2       1.00      0.93      0.96        14

    accuracy                           0.96        54
   macro avg       0.97      0.96      0.96        54
weighted avg       0.96      0.96      0.96        54

Confusion Matrix:
 [[19  0  0]
 [ 1 20  0]
 [ 0  1 13]]
```




```
#single layer perceptron regreesor
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import mean_squared_error

wine = load_wine()
X = wine.data
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

model = Perceptron(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

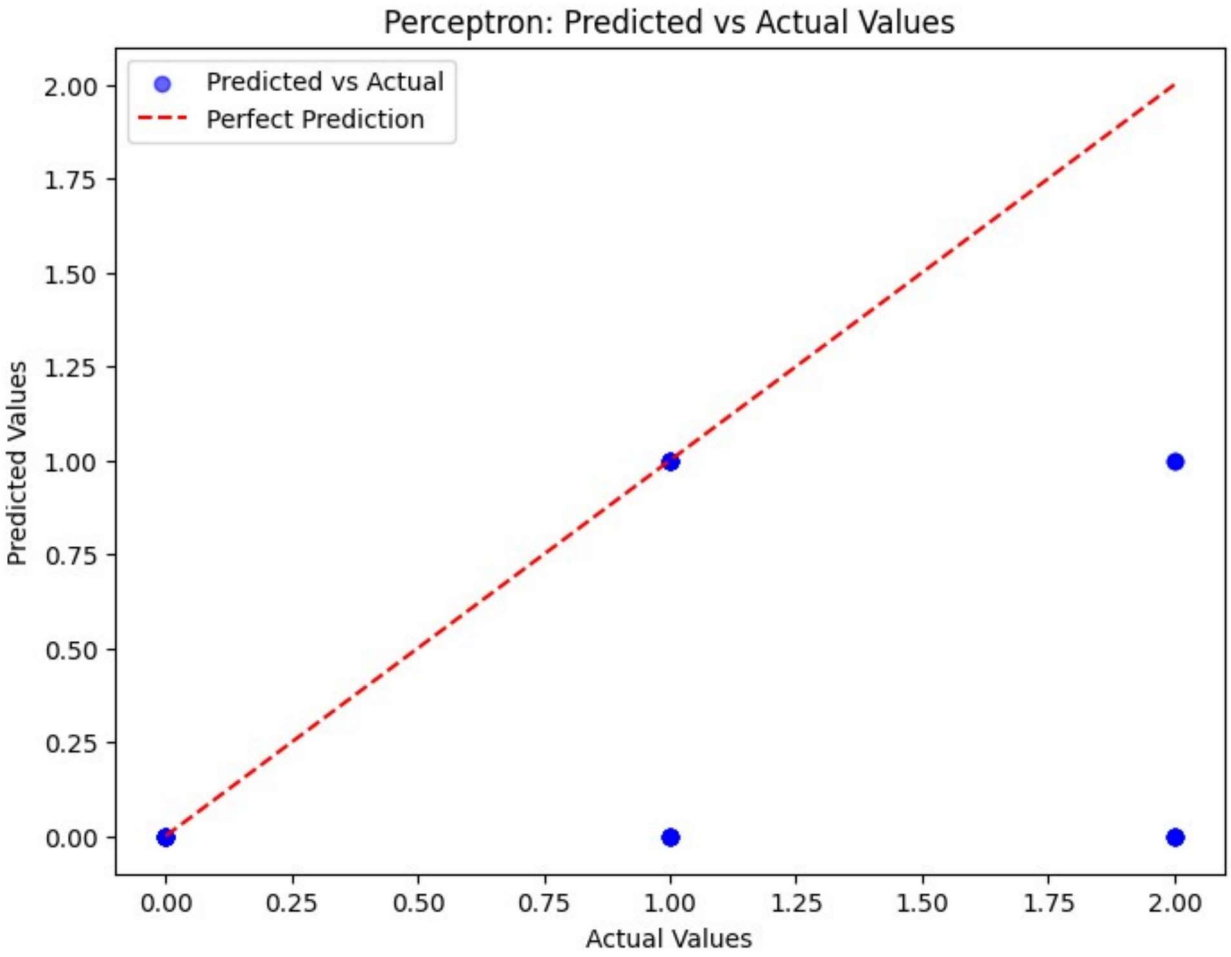
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)

plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.6, label='Predicted
```

```
vs Actual')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],
color='red', linestyle='--', label='Perfect Prediction')
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Perceptron: Predicted vs Actual Values")
plt.legend()
plt.show()
```

```
Mean Squared Error: 1.0
```




```
#Multi layer perceptron
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
```

```python
from sklearn.metrics import accuracy_score, classification_report

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Create and train a neural network with three hidden layers (10, 20,
20 neurons)
model = MLPClassifier(hidden_layer_sizes=(10, 20, 20), max_iter=1000,
random_state=42)
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy Score:", accuracy)

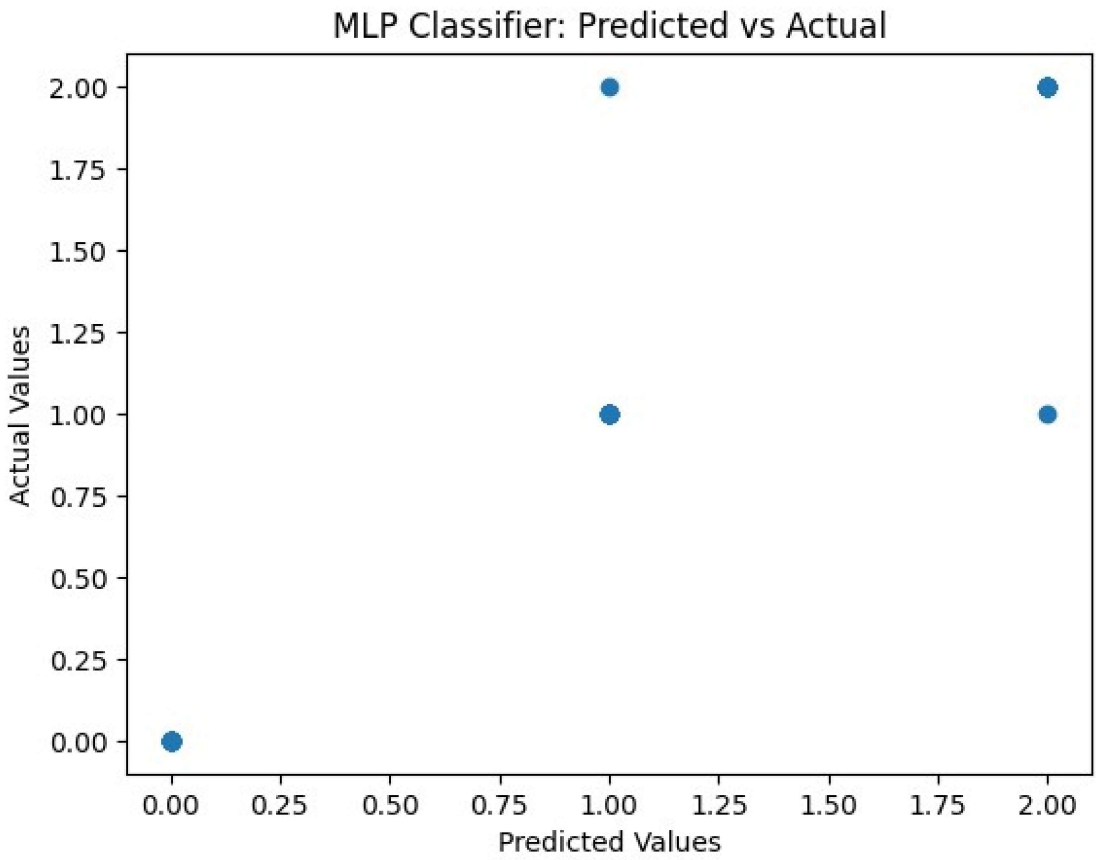
# Print classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Plot the predicted vs actual values
plt.scatter(y_pred, y_test)
plt.xlabel("Predicted Values")
plt.ylabel("Actual Values")
plt.title("MLP Classifier: Predicted vs Actual")
plt.show()
```

```
Accuracy Score: 0.9555555555555556
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       0.92      0.92      0.92        13
           2       0.92      0.92      0.92        13

    accuracy                           0.96        45
   macro avg       0.95      0.95      0.95        45
weighted avg       0.96      0.96      0.96        45
```




```
#Multiple layer Perceptron  classifier
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Load the wine dataset
wine = load_wine()
X = wine.data
y = wine.target

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.3, random_state=42)
```

```
# Create and train a neural network regressor with two hidden layers
model = MLPRegressor(hidden_layer_sizes=(64, 32), max_iter=2000,
solver='adam', activation='relu', random_state=42)
model.fit(X_train, y_train)

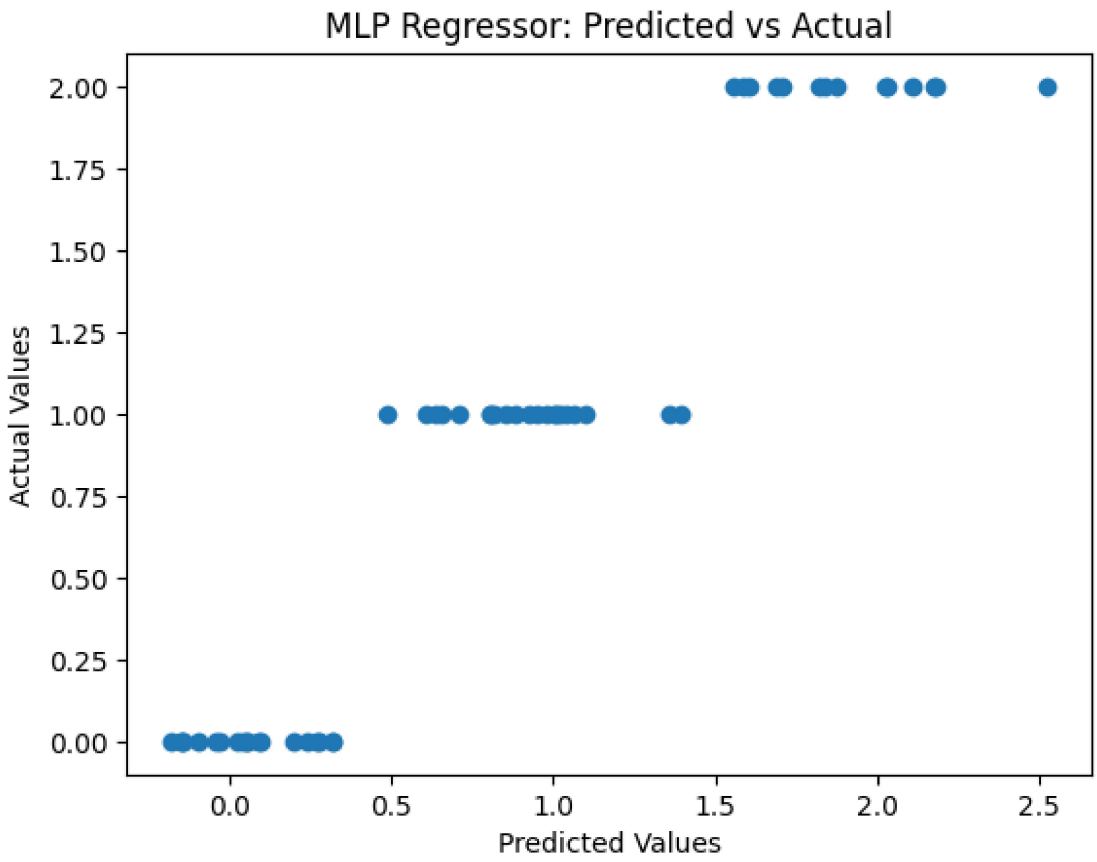
# Make predictions
y_pred = model.predict(X_test)

# Calculate Mean Squared Error and R-squared
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)

# Plot the predicted vs actual values
plt.scatter(y_pred, y_test)
plt.xlabel("Predicted Values")
plt.ylabel("Actual Values")
plt.title("MLP Regressor: Predicted vs Actual")
plt.show()
```

```
Mean Squared Error: 0.05413707438682298
R-squared: 0.9101515600956313
```


MLP Regressor: Predicted vs Actual
Actual Values
Predicted Values
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0.0
0.5
1.0
1.5
2.0
2.5